Rockchip Linux PCIe Developer Guide

ID: RK-KF-YF-141

Release Version: V7.0.0

Release Date: 2026-03-05

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved ©2026 Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

Product Version

Chip Name	Kernel Version
RK1808	4.4, 4.19
RK3528	4.19, 5.10, 6.1,6.12
RK3562	5.10, 6.1,6.12
RK3566/RK3568	4.19, 5.10, 6.1,6.12
RK3572	6.12
RK3576	6.1,6.12
RK3588	5.4, 5.10, 6.1,6.12

RK3399 uses a different PCIe controller IP and is not covered in this document. Please refer to "Rockchip_RK3399_Developer_Guide_PCIe_CN".

Intended Audience

This document (this guide) is primarily intended for the following engineers:

Technical Support Engineers

Software Development Engineers

Revision History

Version	Author	Date	Change Description
V1.0.0	Lin Tao	2021-01-15	Initial version
V1.1.0	Lin Tao	2021-01-22	Added PCIe 3.0 controller exception check information
V1.2.0	Lin Tao	2021-01-26	Added PCIe 2.0 Combo phy exception troubleshooting information
V1.3.0	Lin Tao	2021-02-04	Added MSI and MSI-X support quantity issue description
V1.4.0	Lin Tao	2021-02-05	Added address allocation exception information
V1.5.0	Lin Tao	2021-02-06	Added PCIe2x1 PHY support SSC explanation
V1.6.0	Lin Tao	2021-02-23	Added MSI/MSI-X debugging support and running device exception explanation
V1.7.0	Lin Tao	2021-02-26	Added Legacy INT explanation
V1.8.0	Lin Tao	2021-02-27	Added EP function piece development explanation
V1.9.0	Lin Tao	2021-03-16	Added FW existence exception device explanation

Version	Author	Date	Change Description
V2.0.0	Lin Tao	2021-04-12	Added user mode access exception explanation
V2.1.0	Lin Tao	2021-04-21	Added PCIe to XHCI chip exception explanation
V2.2.0	Lin Tao	2021-04-23	Added lane split reset IO explanation and sleep wake-up exception explanation
V2.3.0	Lin Tao	2021-05-12	Added lane split power supply configuration explanation
V2.4.0	Lin Tao	2021-06-04	Added LTSSM appendix and new debug item explanation
V2.5.0	Lin Tao	2021-06-08	Added PCIe 3.0 driver exit due to PHY exception explanation
V2.6.0	Lin Tao	2021-07-07	Added PCIe address space configuration explanation
V2.7.0	Lin Tao	2021-07-14	Added PCIe device re-scan explanation
V2.7.1	Lin Tao	2021-07-15	Common application problem section format revision
V2.8.0	Lin Tao	2021-07-23	Revised development resource acquisition address
V2.9.0	Lin Tao	2021-07-29	Added legacy interrupt exception and IO port access requirement explanation
V3.0.0	Lin Tao	2021-08-02	Added PCIe 2.0 combo phy charging detection exception handling method
V3.1.0	Lin Tao	2021-08-23	Added PCIe 3.0 power supply configuration impact on clock chip explanation
V3.2.0	Lin Tao	2021-09-01	Added revision of PCIe BAR address space in 64-bit and 32-bit mode explanation
V3.3.0	Lin Tao	2021-09-09	Added FW error log and possible repair methods
V3.4.0	Lin Dingqiang	2021-09-18	Added explanation of device IO BAR address space configuration exception
V3.5.0	Lin Tao	2021-09-23	Added PCIe memory access performance explanation
V3.6.0	Lin Tao	2021-11-03	Added DMA support explanation
V3.7.0	Lin Tao	2021-11-25	Added standard EP bar space exception explanation

Version	Author	Date	Change Description
V4.0.0	Yang Kai	2021-12-20	Added RK3588 explanation
V4.1.0	Lin Tao	2022-01-28	Added present signal, thread initialization explanation and removal of combophy patch explanation
V4.2.0	Lin Dingqiang	2022-03-21	Updated "chip interconnection function" description
V4.3.0	Lin Tao	2022-03-30	Added configurable #PERST reset time explanation
V4.4.0	Lin Dingqiang	2022-05-24	Added PM L1 Substates support explanation
V4.5.0	Lin Tao	2022-07-24	Added debugfs operation explanation
V4.6.0	Lin Tao	2022-07-26	Added platform cache consistency explanation
V4.7.0	Lin Tao	2022-08-08	Added device quirks fix explanation
V4.8.0	Lin Tao	2022-08-17	Added RK1808 explanation, additional debugfs information
V4.9.0	Lin Tao	2022-08-23	Clarified DMA channel situation, added 3588's 5.4 kernel support explanation
V5.0.0	Lin Tao	2022-08-25	Adjusted MEM and IO BAR exception explanation, added M.2 interface explanation, added link failure situation
V5.1.0	Lin Tao	2022-08-28	Added Wi-Fi DTS configuration example and sleep wake-up device status transition exception
V5.2.0	Lin Tao	2022-11-18	Added RK3528 chip explanation and module power-off insufficient troubleshooting
V5.3.0	Lin Tao	2022-12-14	Adjusted standard EP explanation, added PHY clock mode and each chip PCIe controller access priority
V5.4.0	Lin Tao	2023-01-03	Added kernel version, adjusted part of RK356X explanation, added resource filtering method, clarified Lane polarity and line sequence exchange
V5.5.0	Lin Tao	2023-02-06	Fixed RK3528 combo phy clock debug option
V5.6.0	Xue Xiaoming	2023-02-24	Corrected and supplemented BAR address access exception repair measures
V5.7.0	Yang Kai	2023-03-14	Added entering physical signal compliance test mode explanation

Version	Author	Date	Change Description
V5.8.0	Yang Kai	2023-04-18	Removed chip interconnection and standard EP guide, EP mode is explained by <Rockchip_Developer_Guide_PCIE_EP_Standard_Card>
V5.9.0	Lin Tao	2023-05-29	Added NVMe device temperature control explanation
V6.0.0	Lin Tao	2023-06-10	Added explanation for devices not being normally enumerated even though the link is normal
V6.1.0	Lin Tao	2023-06-25	Added PCIe device wake-up explanation
V6.2.0	Lin Dingqiang	2023-07-05	Added DMATEST, added several common application explanations
V6.3.0	Xiao Yao	2023-07-31	Corrected PCIe WiFi REG ON pin configuration explanation
V6.4.0	Lin Tao	2023-08-09	Unified compliance test configuration specification
V6.5.0	Lin Dingqiang	2023-08-15	Modified to use the new version of DMATEST command
V6.6.0	Lin Tao	2023-12-19	Corrected msi-map explanation
V6.7.0	Yang Kai	2024-01-12	Added low-speed IO signal explanation
V6.8.0	Lin Tao	2024-02-26	Updated kernel version and new chip support
V6.9.0	Lin Dingqiang	2024-05-25	Updated domain address allocation example
V6.10.0	Lin Dingqiang	2024-07-25	Optimized Lane reversal/Lane polarity explanation
V6.11.0	Lin Tao	2024-08-01	Added explanation based on GPIO mode plugging and unplugging, further clarified Lane reversal usage
V6.12.0	Lin Tao	2024-08-22	Added explanation about downstream device binding ID issues
V6.13.0	Lin Tao	2024-09-04	Added PCIe PMU perf support explanation
V6.14.0	Lin Tao	2024-09-07	Independent stability statistics section, added error injection explanation
V6.15.0	Lin Tao	2024-10-18	Supplemented board-level configurable timing

Version	Author	Date	Change Description
V6.16.0	Lin Tao	2024-11-05	Supplemented NVMe exception handling
V6.17.0	Lin Tao	2024-11-19	Added device sleep without power cut configuration; other configuration fixes
V6.18.0	Lin Tao	2024-12-03	Added PC model card death problem explanation
V6.19.0	Lin Tao	2025-01-12	Added some performance explanations and BAR access explanations
V6.20.0	Lin Tao, Lin Dingqiang	2025-02-07	Supplemented configuration to limit PCIe bandwidth, added non-coherent clock scheme explanation
V6.20.1	Lin Dingqiang	2025-02-14	Modified reference to PCIE_EP_Standard_Card document
V6.21.0	Lin Tao	2025-02-18	Supplemented appendix on PCIe bus transmission status query and AI card issues
V6.22.0	Lin Tao	2025-02-26	Supplemented appendix on specific types of PCIe transmission blockage and controller base address
V6.23.0	Lin Tao	2025-03-14	Removed PC model card death explanation
V6.24.0	Lin Tao	2025-03-17	Added explanation for link maintenance in PCIe sleep state
V6.25.0	Lin Tao, Lin Dingqiang	2025-05-01	Migrated performance issues to other documents, removed redundancy; added iommu usage introduction; added PHY tuning explanation
V6.26.0	Lin Tao	2025-05-22	Add RK3588 bifurcation note and AER injection support
V6.27.0	Lin Tao	2025-07-29	amend IO label; Note for increasing 32-bit BAR; Add more PCIe2 failure; Add NVMe recovery patch
V6.28.0	Lin Tao	2025-09-05	Revise the description of the L1 substate to avoid ambiguity; Revise and remove the description about MSI; delete the explanation of BAR for each individual chip
V6.29.0	Lin Tao, Lin Dingqiang	2025-11-20	Amend AER info, Add suggestions for troubleshooting low-speed I/O during link training failures
V6.30.0	Lin Tao	2025-12-20	Fixed L1SS abnormal configuration check; Kernel version update; GKI check
V6.31.0	Lin Tao	2026-01-27	Add RK3572; Remove the chip enumeration and other content from the FAQ sections "Allocatable MSI Count" and "Cache Coherence Handling."

Version	Author	Date	Change Description
V7.0.0	Lin Tao	2026-03-05	Split the document. Keep only resource and function configuration instructions, remove troubleshooting sections.

Table of Contents

Rockchip Linux PCIe Developer Guide

1. Chip Resource Introduction
    - 1.1 RK1808
    - 1.2 RK3528
    - 1.3 RK3562
    - 1.4 RK3566
    - 1.5 RK3568
        - 1.5.1 Controller
        - 1.5.2 PHY
    - 1.6 RK3572
    - 1.7 RK3576
    - 1.8 RK3588
        - 1.8.1 Controller
        - 1.8.2 PHY
    - 1.9 RK3588S
        - 1.9.1 Controller
        - 1.9.2 PHY
2. DTS Configuration
    - 2.1 Configuration Key Points
    - 2.2 RK1808 DTS Configuration
    - 2.3 RK3528 DTS Configuration
    - 2.4 RK356X DTS Configuration
        - 2.4.1 RK3562 dts
        - 2.4.2 RK3566 dts
        - 2.4.3 RK3568 dts
        - 2.4.4 RK3572 dts
        - 2.4.5 RK3576 dts
    - 2.5 RK3588 DTS Configuration
        - 2.5.1 Example 1 PCIe 3.0 4-Lane Root Complex (RC) + 2 PCIe 2.0 (comboPHY) (RK3588 EVB1)
        - 2.5.2 Example 2 PCIe 3.0 PHY Split into 2 2-Lane RC, 3 PCIe 2.0 1-Lane (comboPHY)
        - 2.5.3 Example 3: PCIe 3.0 PHY Split into 4 Single Lanes, One Using PCIe 2.0 Single Lane (Combo PHY)
        - 2.5.4 Enable IOMMU Support
    - 2.6 DTS Property Description
        - 2.6.1 Common Controller DTS Configurations
        - 2.6.2 comboPHY dts Configuration
        - 2.6.3 PCIe30 PHY DTS Configuration
    - 2.7 Fill DTS Based on Schematic Diagram
        - 2.7.1 Low-Speed IO Description
        - 2.7.2 DTS Configuration Method
    - 2.8 Wi-Fi Module Device Tree Writing Example
3. menuconfig Configuration
4. Standard EP Function Development
5. RC mode PM L1 Substates Support
6. GPIO-Based Detection Mechanism for Plug and Unplug Events
    - 6.1 Hardware Requirements
    - 6.2 Software Requirements
    - 6.3 Usage Restrictions
7. Kernel DMATEST
8. Kernel Stability Statistics
9. Kernel AER support and Injection Test Support
10. Kernel Error Injection Test Support
11. Kernel PMU perf Support
    - 11.1 Software and Configuration
    - 11.2 Usage Instructions
12. Appendix

12.1 Debugfs Exported Information Analysis Table
12.2 Error Injection Configuration Comparison Table
12.3 PCIe TX Emphasis Preset Value table
12.4 Development Resource Acquisition Address
12.5 Detailed Explanation of PCIe Address Space Configuration
12.6 Board-Level Configurable Timing
12.7 RK3568 PCIe30PHY SRNS Patch

1. Chip Resource Introduction

For the address ranges of BAR (Base Address Register) spaces that can be allocated by each controller within the chip, please refer to the *Internal Address Mapping* section under the PCIe chapter in the respective chip's TRM (Technical Reference Manual). The keyword to look for is `PCIe Outbound Memory`. Each controller's corresponding table includes the starting address (Base Address) and total size (Address Length) for both 32-bit BARs and 64-bit BARs. Note: The peripheral configuration space, I/O space, and memory (MEM) space share this address segment, and the MEM space does not support prefetching.

1.1 RK1808

Resource	Mode	Lane Splitting	DMA	IOMMU	HW cache coherent	ASPM	Remarks	MSI
PCIe Gen2 x 2 lane	RC/EP	No	2 read Channels + 2 write Channels	No	No	L0s/L1	Internal Clock	65535

1.2 RK3528

Resource	Mode	Lane Splitting	DMA	IOMMU	HW cache coherent	ASPM	Remarks	MSI
PCIe Gen2 x 1 lane	RC only	No	No	No	No	ALL	Internal Clock	32

1.3 RK3562

Resource	Mode	Lane Splitting	DMA	IOMMU	HW cache coherent	ASPM	Remarks	MSI
PCIe Gen2 x 1 lane	RC only	No	No	No	No	ALL	Internal Clock	32

1.4 RK3566

Resource	Mode	Lane Splitting	DMA	IOMMU	HW cache coherent	ASPM	Remarks	MSI
PCIe Gen2 x 1 lane	RC only	No	No	No	No	L0s/L1	Internal Clock	65535

1.5 RK3568

1.5.1 Controller

Resource	Mode	Lane Splitting	DMA	IOMMU	HW cache coherent	ASPM	Remarks	MSI
PCIe Gen2 x 1 lane	RC only	No	No	No	No	L0s/L1	Internal Clock	65535
PCIe Gen3 x 2 lane	RC/EP	1 lane RC + 1 lane RC	2 Read Channels + 2 Write Channels	No	No	ALL	External Oscillator, Only Supports pcie30phy	65535
PCIe Gen3 x 1 lane	RC	1 lane RC	No	No	No	ALL	External Oscillator, Only Supports pcie30phy	65535

1.5.2 PHY

Resource	DTS Node	Reference Clock	Splitting	Combo Capability
pcie30phy	phy@fe8c0000	External	2Lane: Default 1Lane + 1Lane: rockchip, bifurcation	PCIe Exclusive
combphy2_psq	phy@fe840000	Internal/External	-	Combo with SATA/RGMII

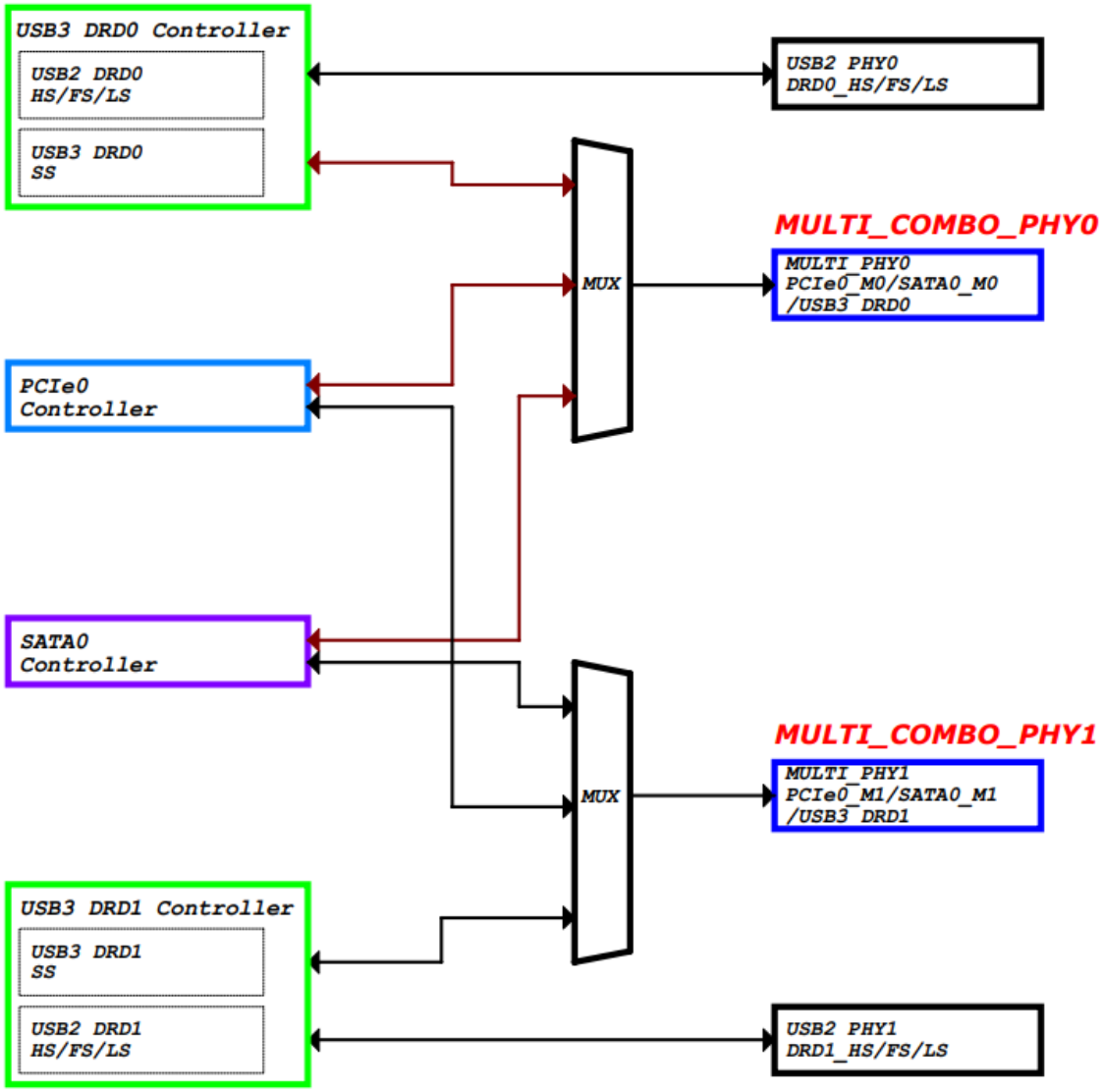
Explanation:

- pcie30phy 2Lane default configuration is PCIe Gen3 x 2 lane, after splitting, "PCIe Gen3 x 2 lane" and "PCIe Gen3 x 1 lane" controllers each have 1Lane.
- pcie30phy supports the separate refclk architecture of SRNS No SSC, configuration reference to the appendix "RK3568 pcie30phy SRNS Patch" section, it is recommended to ensure signal consistency through standard signal measurement kits and peripheral stress testing.

1.6 RK3572

Resource	Mode	Lane Splitting Supported	DMA Supported	IOMMU Supported	HW cache coherent Supported	ASPM Supported	Remarks	MSI	PTM Supported
PCIe Gen2 x 1 lane	RC only	No	No	Yes	Yes	ALL	Internal Clock	32	Yes
PCIe Gen2 x 1 lane	RC only	No	No	Yes	Yes	ALL	Internal Clock	32	Yes

The PCIe0 controller of the RK3572 supports routing to either COMBO_PHY0 or COMBO_PHY1. The software configuration of routing only requires modifying the Device Tree Source (DTS) to ensure that the PCIe0 node references the correct PHY node.



1.7 RK3576

Resource	Mode	Lane Splitting Supported	DMA Supported	IOMMU Supported	HW cache coherent Supported	ASPM Supported	Remarks	MSI
PCIe Gen2 x 1 lane	RC only	No	No	Yes	Yes	ALL	Internal Clock	32
PCIe Gen2 x 1 lane	RC only	No	No	Yes	Yes	ALL	Internal Clock	32

1.8 RK3588

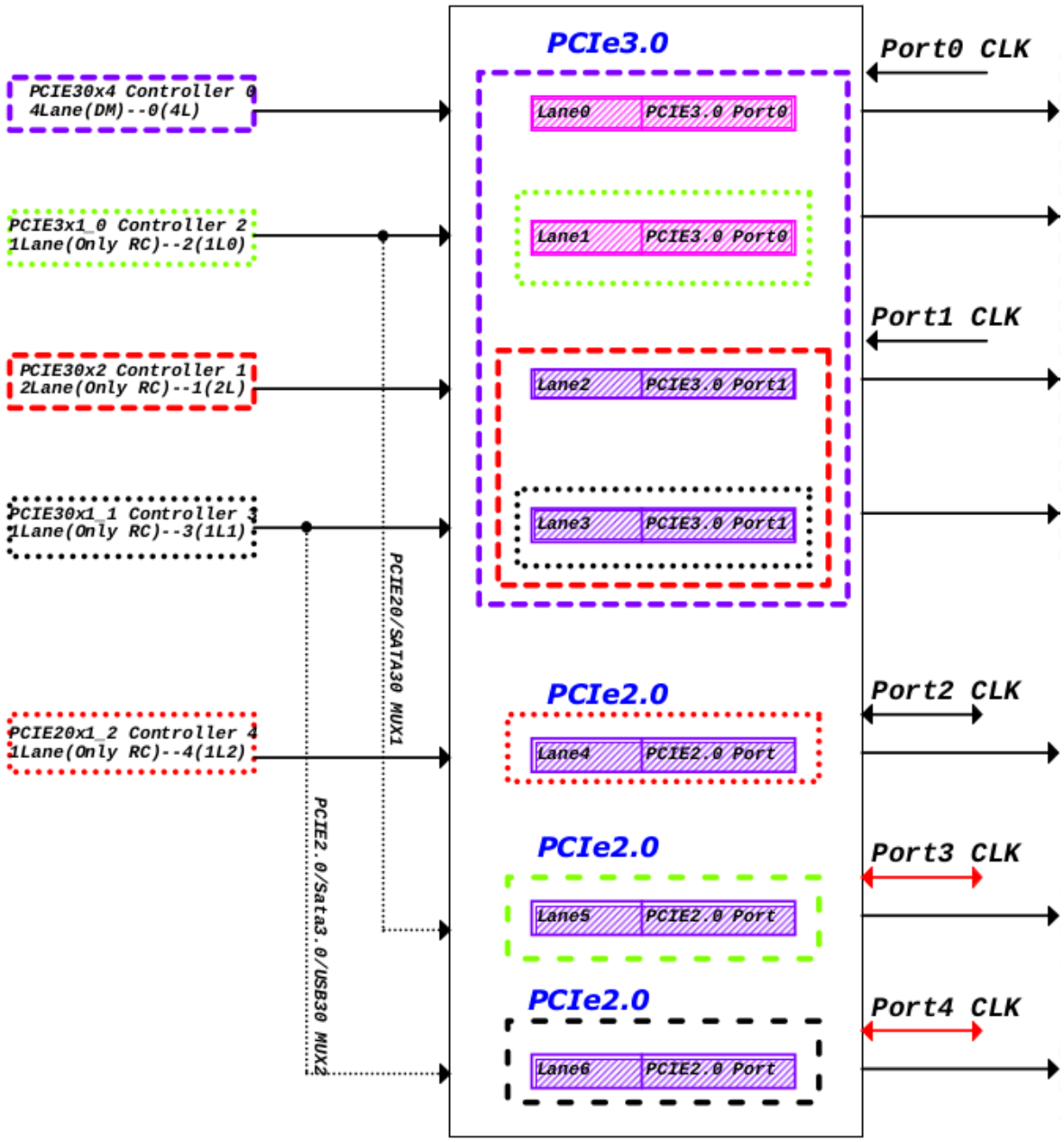
The RK3588 features five PCIe controllers, which share the same hardware IP but have different configurations. One of the 4-lane DM mode controllers can be used as an Endpoint (EP), while the other 2-lane and three 1-lane controllers can only function as Root Complexes (RC).

The RK3588 is equipped with two types of PCIe PHYs. One type is the PCIe 3.0 PHY, which includes two Ports with a total of four Lanes. The other type is the PCIe 2.0 PHY, which consists of three units, each with a single Lane, and they are used in conjunction with SATA and USB.

The 4-lane PCIe 3.0 PHY can be split according to actual requirements. After splitting, the corresponding controllers need to be properly configured, and all configurations are completed in the Device Tree Source (DTS) without the need to modify the driver.

Usage limitations:

1. After splitting the PCIe 3.0 PHY, the PCIe 3.0 x4 controller can only be fixedly paired with the port0 of the PCIe 3.0 PHY when operating in 2-lane mode, and can only be fixedly paired with port0 lane0 when operating in 1-lane mode;
2. After splitting the PCIe 3.0 PHY, the PCIe 3.0 x2 controller can only be fixedly paired with the port1 of the PCIe 3.0 PHY when operating in 2-lane mode, and can only be fixedly paired with port1 lane0 when operating in 1-lane mode;
3. When the PCIe 3.0 PHY is split into four 1-lane units, the port0 lane1 of the PCIe 3 PHY can only be fixedly paired with the PCIe 2 x1 l0 controller, and the port1 lane1 can only be fixedly paired with the PCIe 2 x1 l1 controller;
4. The PCIe 3.0 x4 controller operating in EP mode can use the 4-lane mode or the 2-lane mode with the port0 of the PCIe 3.0 PHY, and the two lanes of port1 of the PCIe 3.0 PHY can be used as RC to cooperate with other controllers. By default, when using a common clock as the reference clock, it is not possible for the lane0 of the PCIe 3.0 PHY port0 to operate in EP mode while lane1 operates in RC mode to cooperate with other controllers, because the two lanes of Port0 share a common input reference clock, and the simultaneous use of the clock by RC and EP may cause conflicts.
5. If only one port of the RK3588 PCIe 3.0 PHY is used, the other port still needs to be powered, and other signals such as refclk can be grounded.



1.8.1 Controller

Resource	Mode	DTS Node	Available PHY	Internal DMA	Supported ASPM	Supported IOMMU	Supported HW cache coherent	MSI
PCIe Gen3 x 4 lane	RC/EP	pcie3x4: pcie@fe150000	pcie30phy	2 read Channels + 2 write Channels	ALL	Yes	No	63353
PCIe Gen3 x 2 lane	RC only	pcie3x2: pcie@fe160000	pcie30phy	No	ALL	Yes	No	63353
PCIe Gen3 x 1 lane	RC only	pcie2x1l0: pcie@fe170000	pcie30phy, combphy1_ps	No	ALL	Yes	No	63353
PCIe Gen3 x 1 lane	RC only	pcie2x1l1: pcie@fe180000	pcie30phy, combphy2_psu	No	ALL	Yes	No	63353
PCIe Gen3 x 1 lane	RC only	pcie2x1l2: pcie@fe190000	combphy0_ps	No	ALL	Yes	No	63353

1.8.2 PHY

Resource	DTS Node	Reference Clock	Splitting	Combo Support
pcie30phy	phy@fee80000	External	4Lane*1: PHY_MODE_PCIE_AGGREGATION 2Lane+2Lane: PHY_MODE_PCIE_NANBNB 2Lane+1Lane+1Lane: PHY_MODE_PCIE_NANBBI 1Lane*4: PHY_MODE_PCIE_NABIBI	PCIe Specific
combphy0_ps	phy@fee00000	Internal/External	-	Combined with SATA
combphy1_ps	phy@fee10000	Internal/External	-	Combined with SATA
combphy2_psu	phy@fee20000	Internal/External	-	Combined with SATA/USB3

Explanation:

- pcie30phy does not support separate refclk architecture.

1.9 RK3588S

The PCIe interface of RK3588S is relatively straightforward, featuring 2 single-lane controllers and 2 single-lane comboPHYs that can be utilized for PCIe 2.0, with a one-to-one correspondence relationship.

1.9.1 Controller

Resource	Mode	DTS Node	Available PHY	Internal DMA	ASPM	IOMMU	HW cache coherent	MSI
PCIe Gen3 x 1 lane	RC only	pcie2x1l1: pcie@fe180000	combphy2_psu	No	ALL	Yes	No	65535
PCIe Gen3 x 1 lane	RC only	pcie2x1l2: pcie@fe190000	combphy0_ps	No	ALL	Yes	No	65535

1.9.2 PHY

Resource	DTS Node	Reference Clock	Split	Combo with SATA/USB3
combphy0_ps	phy@fee00000	Internal/External	-	Combined with SATA
combphy2_psu	phy@fee20000	Internal/External	-	Combined with SATA/USB3

2. DTS Configuration

2.1 Configuration Key Points

The configuration of PCIe is mostly fixed, and there are not many variables that need to be configured at the board level DTS. Refer to the following key points for configuration:

1. Controller/PHY Enable: After determining the scheme, enable the correct controller and PHY according to the schematic diagram. Note that the index of the controller and PHY may not be sequentially matched, such as RK3588's PCIe2x1l0 does not correspond to combphy0_ps;
2. Controller: Some controllers (such as RK3588's PCIe2x1l0 and PCIe2x1l1) have more than one PHY option, configure the "phys" correctly according to the scheme design;
3. Controller: As RC, it is usually necessary to configure "reset-gpios", which corresponds to the "PERSTn" signal of PCIE in the schematic diagram;
4. Controller: As RC, it may be necessary to configure "vpcie3v3-supply", which corresponds to the fixed regulator controlled by the "PWREN" gpio signal of PCIE;
5. Controller: When used as EP, it is necessary to modify "compatible" to the corresponding string for EP mode;
6. PHY: The PCIe30phy has a total of 4 lanes, which can be used separately, and it is necessary to correctly configure the "rockchip,pcie30-phymode" mode according to the scheme.

2.2 RK1808 DTS Configuration

The DTS configuration for RK1808, all implementation modes have examples in the SDK's evb code that can be referenced. You can copy the matching content according to the patterns in the table below to the product board-level dts for use.

Resource	Mode	Reference Configuration	Controller Node	PHY Node	Remarks
PCIe Gen2 x 2 lane	RC	rk1808-evb.dtsi	pcie0@fc400000	combphy	Need to disable usbdrd_dwc3 and usbdrd3
PCIe Gen2 x 2 lane	EP	Add compatible = "rockchip,rk1808-pcie-ep", "snps,dw-pcie" to the pcie0 node in rk1808-evb.dtsi;	pcie0@fc400000	combphy	Need to disable usbdrd_dwc3 and usbdrd3

2.3 RK3528 DTS Configuration

The DTS configuration for RK3528, all implementation modes have examples in the SDK's evb code for reference, and you can copy the matching content according to the modes listed in the table below to use in the product board-level dts.

Resource	Mode	Reference Configuration	Controller Node	PHY Node	Note
PCIe Gen2 x 1 lane	RC	rk3528-evb2-ddr3-v10.dtsi	pcie2x1@fe4f0000	combphy_pu	-

2.4 RK356X DTS Configuration

The DTS configuration for RK356x, all implementation modes have examples in the SDK's evb code for reference, you can select the matching content according to the pattern in the table below and copy it to the product board-level DTS for use.

2.4.1 RK3562 dts

Resource	Mode	Reference Configuration	Controller Node	PHY Node
PCIe Gen2 x 1 lane	RC	rk3562-evb1-lp4x-v10.dtsi	pcie2x1@ff500000	combphy_pu

2.4.2 RK3566 dts

Resource	Mode	Reference Configuration	Controller Node	PHY Node
PCIe Gen2 x 1 lane	RC	rk3566-evb1-ddr4-v10.dtsi	pcie2x1@fe260000	combphy2_psq

2.4.3 RK3568 dts

Resource	Mode	Reference Configuration	Controller Node	PHY Node
PCIe Gen2 x 1 lane	RC	rk3568-evb2-lp4x-v10.dtsi	pcie2x1@fe260000	combphy2_psq
PCIe Gen3 x 2 lane	RC	rk3568-evb1-ddr4-v10.dtsi	pcie3x2@fe280000	pcie30phy
PCIe Gen3 Split 1 lane + 1 lane	RC	rk3568-evb6-ddr3-v10.dtsi	pcie3x2@fe280000 pcie3x1@fe270000	pcie30phy
PCIe Gen3 x 2 lane	EP	rk3568-iotest-ddr3-v10.dts	pcie3x2@fe280000	pcie30phy

2.4.4 RK3572 dts

Resource	Mode	Reference Configuration	Controller Node	PHY Node
PCIe Gen2 x 1 lane	RC	rk3572-evb1-v10.dtsi	pcie0@29d00000	combphy0_ps or combphy1_psu
PCIe Gen2 x 1 lane	RC	rk3572-evb2-v10.dtsi	pcie1@29d10000	combphy2_ps

2.4.5 RK3576 dts

Resource	Mode	Reference Configuration	Controller Node	PHY Node
PCIe Gen2 x 1 lane	RC	rk3576-test1.dtsi	pcie0@2a200000	combphy0_ps
PCIe Gen2 x 1 lane	RC	rk3576-test1.dtsi	pcie1@2a210000	combphy1_psu

2.5 RK3588 DTS Configuration

The RK3588 controller and PHY have numerous combinations, which cannot be exhaustively listed in the SDK's evb code. Configuration can be done based on key points, and several typical examples are provided here for reference.

Note: If splitting a PCIe 3.0 PHY, the enable pin of the reference clock chip for the 3.0 PHY must be controlled collectively by all split-out controllers via the power supply node. This can be achieved by:

1. Having all controllers share a common vpcie3v3-supply reference, or
2. Configuring the vpcie3v3-supply node with the properties regulator-always-on and regulator-boot-on. Otherwise, the enable signal should be hardwired to an "always enabled" state in the hardware design.

2.5.1 Example 1 PCIe 3.0 4-Lane Root Complex (RC) + 2 PCIe 2.0 (comboPHY) (RK3588 EVB1)

```
/ {
    vcc3v3_pcie30: vcc3v3-pcie30 {
        compatible = "regulator-fixed";
        regulator-name = "vcc3v3_pcie30";
        regulator-min-microvolt = <3300000>;
        regulator-max-microvolt = <3300000>;
        enable-active-high;
        gpios = <&gpio3 RK_PC3 GPIO_ACTIVE_HIGH>;
        startup-delay-us = <5000>;
        vin-supply = <&vcc12v_dcin>;
    };
};

&combphy1_ps {
    status = "okay";
};

&combphy2_psu {
    status = "okay";
};

&pcie2x1l0 {
    phys = <&combphy1_ps PHY_TYPE_PCIE>;
    reset-gpios = <&gpio4 RK_PA5 GPIO_ACTIVE_HIGH>;
    status = "okay";
};

&pcie2x1l1 {
    phys = <&combphy2_psu PHY_TYPE_PCIE>;
    reset-gpios = <&gpio4 RK_PA2 GPIO_ACTIVE_HIGH>;
    pinctrl-names = "default";
    pinctrl-0 = <&rtl8111_isolate>;
    status = "okay";
};

&pcie30phy {
    rockchip,pcie30-phymode = <PHY_MODE_PCIE_AGGREGATION>;
    status = "okay";
};

&pcie3x4 {
    reset-gpios = <&gpio4 RK_PB6 GPIO_ACTIVE_HIGH>;
    vpcie3v3-supply = <&vcc3v3_pcie30>;
    status = "okay";
};
```

2.5.2 Example 2 PCIe 3.0 PHY Split into 2 2-Lane RC, 3 PCIe 2.0 1-Lane (comboPHY)

```
/ {
    vcc3v3_pcie30: vcc3v3-pcie30 {
        compatible = "regulator-fixed";
        regulator-name = "vcc3v3_pcie30";
        regulator-min-microvolt = <3300000>;
        regulator-max-microvolt = <3300000>;
        enable-active-high;
        gpios = <&gpio3 RK_PC3 GPIO_ACTIVE_HIGH>;
        startup-delay-us = <5000>;
        vin-supply = <&vcc12v_dcin>;
    };
};

&combphy0_ps {
    status = "okay";
};

&combphy1_ps {
    status = "okay";
};

&combphy2_psu {
    status = "okay";
};

&pcie2x1l0 {
    phys = <&combphy1_ps PHY_TYPE_PCIE>;
    reset-gpios = <&gpio4 RK_PA5 GPIO_ACTIVE_HIGH>;
    vpcie3v3-supply = <&vcc3v3_pcie30>;
    status = "okay";
};

&pcie2x1l1 {
    phys = <&combphy2_psu PHY_TYPE_PCIE>;
    reset-gpios = <&gpio4 RK_PA2 GPIO_ACTIVE_HIGH>;
    vpcie3v3-supply = <&vcc3v3_pcie30>;
    status = "okay";
};

&pcie2x1l2 {
    reset-gpios = <&gpio4 RK_PC1 GPIO_ACTIVE_HIGH>;
    vpcie3v3-supply = <&vcc3v3_pcie30>;
    status = "okay";
};

&pcie30phy {
    rockchip,pcie30-phymode = <PHY_MODE_PCIE_NANBNB>;
    status = "okay";
};

&pcie3x2 {
    reset-gpios = <&gpio4 RK_PB0 GPIO_ACTIVE_HIGH>;
    vpcie3v3-supply = <&vcc3v3_pcie30>;
```

```
    status = "okay";
};

&pcie3x4 {
    num-lanes = <2>;
    reset-gpios = <&gpio4 RK_PB6 GPIO_ACTIVE_HIGH>;
    vpcie3v3-supply = <&vcc3v3_pcie30>;
    status = "okay";
};
```

2.5.3 Example 3: PCIe 3.0 PHY Split into 4 Single Lanes, One Using PCIe 2.0 Single Lane (Combo PHY)

NOTE:

- Hardware-wise, pcie2x1l0/pcie2x1l1 interfaces with PCIe 3.0 PHY corresponding TX/RX signals, and the dts disables combphy1_ps/combphy2_psu nodes.

Reference Code:

```
/ {
    vcc3v3_pcie30: vcc3v3-pcie30 {
        compatible = "regulator-fixed";
        regulator-name = "vcc3v3_pcie30";
        regulator-min-microvolt = <3300000>;
        regulator-max-microvolt = <3300000>;
        enable-active-high;
        gpios = <&gpio3 RK_PC3 GPIO_ACTIVE_HIGH>;
        startup-delay-us = <5000>;
        vin-supply = <&vcc12v_dcin>;
    };
};

&combphy0_ps {
    status = "okay";
};

&pcie2x1l0 {
    phys = <&pcie30phy>;
    reset-gpios = <&gpio4 RK_PA5 GPIO_ACTIVE_HIGH>;
    vpcie3v3-supply = <&vcc3v3_pcie30>;
    status = "okay";
};

&pcie2x1l1 {
    phys = <&pcie30phy>;
    reset-gpios = <&gpio4 RK_PA2 GPIO_ACTIVE_HIGH>;
    vpcie3v3-supply = <&vcc3v3_pcie30>;
    status = "okay";
};

&pcie2x1l2 {
    reset-gpios = <&gpio4 RK_PC1 GPIO_ACTIVE_HIGH>;
    vpcie3v3-supply = <&vcc3v3_pcie30>;
    status = "okay";
};
```

```
&pcie30phy {
    rockchip,pcie30-phymode = <PHY_MODE_PCIE_NABIBI>;
    status = "okay";
};

&pcie3x2 {
    num-lanes = <1>;
    reset-gpios = <&gpio4 RK_PB0 GPIO_ACTIVE_HIGH>;
    vpcie3v3-supply = <&vcc3v3_pcie30>;
    status = "okay";
};

&pcie3x4 {
    num-lanes = <1>;
    reset-gpios = <&gpio4 RK_PB6 GPIO_ACTIVE_HIGH>;
    vpcie3v3-supply = <&vcc3v3_pcie30>;
    status = "okay";
};
```

2.5.4 Enable IOMMU Support

- Add PCIe IOMMU Configuration

```
# Enable PCIe IOMMU
--- a/arch/arm64/boot/dts/rockchip/rk3588s.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3588s.dtsi
@@ -3370,7 +3370,7 @@
mmu600_pcie: iommu@fc900000 {
    compatible = "arm,smmu-v3";
    reg = <0x0 0xfc900000 0x0 0x200000>;
    interrupts = <GIC_SPI 369 IRQ_TYPE_LEVEL_HIGH>,
                 <GIC_SPI 371 IRQ_TYPE_LEVEL_HIGH>,
                 <GIC_SPI 374 IRQ_TYPE_LEVEL_HIGH>,
                 <GIC_SPI 367 IRQ_TYPE_LEVEL_HIGH>;
                 interrupt-names = "eventq", "gerror", "priq", "cmdq-sync";
                 #iommu-cells = <1>;
-                status = "disabled";
+                status = "okay";
};

# Controllers requiring IOMMU should reference iommu-map as needed, ensuring
numerical matching with msi-map
--- a/arch/arm64/boot/dts/rockchip/rk3588.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3588.dtsi
@@ -548,6 +548,7 @@
                 num-viewport = <8>;
                 max-link-speed = <3>;
                 msi-map = <0x0000 &its1 0x0000 0x1000>;
+                iommu-map = <0x0000 &mmu600_pcie 0x0000 0x1000>;
                 num-lanes = <4>;
                 phys = <&pcie30phy>;
                 phy-names = "pcie-phy";
@@ -603,6 +604,7 @@
                 num-viewport = <8>;
```

```
                max-link-speed = <3>;
                msi-map = <0x1000 &its1 0x1000 0x1000>;
+               iommu-map = <0x1000 &mmu600_pcie 0x1000 0x1000>;
                num-lanes = <2>;
                phys = <&pcie30phy>;
                phy-names = "pcie-phy";
@@ -658,6 +660,7 @@
                num-viewport = <4>;
                max-link-speed = <2>;
                msi-map = <0x2000 &its0 0x2000 0x1000>;
+               iommu-map = <0x2000 &mmu600_pcie 0x2000 0x1000>;
                num-lanes = <1>;
                phys = <&combphy1_ps PHY_TYPE_PCIE>;
                phy-names = "pcie-phy";
diff --git a/arch/arm64/boot/dts/rockchip/rk3588s.dtsi
b/arch/arm64/boot/dts/rockchip/rk3588s.dtsi
index 9e19b61..b77a073 100644
--- a/arch/arm64/boot/dts/rockchip/rk3588s.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3588s.dtsi
@@ -6319,6 +6319,7 @@
                num-viewport = <4>;
                max-link-speed = <2>;
                msi-map = <0x3000 &its0 0x3000 0x1000>;
+               iommu-map = <0x3000 &mmu600_pcie 0x3000 0x1000>;
                num-lanes = <1>;
                phys = <&combphy2_psu PHY_TYPE_PCIE>;
                phy-names = "pcie-phy";
@@ -6373,6 +6374,7 @@
                num-viewport = <4>;
                max-link-speed = <2>;
                msi-map = <0x4000 &its0 0x4000 0x1000>;
+               iommu-map = <0x4000 &mmu600_pcie 0x4000 0x1000>;
                num-lanes = <1>;
                phys = <&combphy0_ps PHY_TYPE_PCIE>;
                phy-names = "pcie-phy";
```

- Obtain the Kernel5.10_PCIe_MMU_support.zip or Kernel6.1_PCIe_MMU_support.zip from the "Development Resource Acquisition Address" mentioned in this document's appendix, and apply the patches included in the compressed package. Note: Due to the early Linux kernel's inability to support multiple versions of the IOMMU system simultaneously, this patch has been customized for modification.
- If display graphics and multimedia-related IPs are not required, the patches can be omitted, and the `CONFIG_ROCKCHIP_IOMMU` config item can be disabled.

2.6 DTS Property Description

2.6.1 Common Controller DTS Configurations

1. compatible = <?>;

Optional Configuration Item: This item sets whether the PCIe interface is using RC mode or EP mode.

For RK3568 functioning as an RC, it needs to be configured as compatible = "rockchip,rk3568-pcie", "snps,dw-pcie"; and if it needs to be changed to EP mode, it should be modified to compatible = "rockchip,rk3568-pcie-ep", "snps,dw-pcie"; similarly for RK1808, RK3588, just replace the rk3568 field with rk1808 and rk3588 respectively.

2. reset-gpios = <&gpio3 13 GPIO_ACTIVE_HIGH>;

Mandatory Configuration Item: This item sets the PERSTN reset signal for the PCIe interface; whether it is a slot or a soldered device, please find the pin on the schematic and configure it correctly. Otherwise, it is very likely that the link establishment will not be stable. Also, a special reminder, if multiple lanes of the PCIe interface are split, each split node must be configured with a different PERSTN signal line.

3. `num-lanes = <4>;`

Optional Configuration Item: This configuration sets the number of lanes used by the PCIe device, which has been configured in the chip-level dtsi by default, and does not need to be adjusted. When used in combination with a splittable phy, it is recommended to configure according to the actual hardware.

4. `max-link-speed = <2>;`

Optional Configuration Item: This configuration sets the bandwidth version of PCIe, 1 indicates Gen1, 2 indicates Gen2, and 3 indicates Gen3. It should be noted that this configuration is related to the chip, and in principle, it does not need to be configured for each board, so we have already configured it in the chip-level dtsi, just as a test means, or a downgrade means after the customer's board design is abnormal.

5. `status = <okay>;`

Mandatory Configuration Item: This configuration needs to be enabled at the same time in the pcie controller node and the corresponding phy node.

6. `vpcie3v3-supply = <&vdd_pcie3v3>;`

Optional Configuration Item: Used to configure the 3V3 power supply for the PCIe peripheral (in principle, our company's hardware reference schematic combines the 12V power supply and the 3V3 power supply of the PCIe slot for control, so after configuring the 3v3 power supply, the 12V power supply is controlled together). If the board level needs to control the 3V3 of the PCIe peripheral, define a corresponding regulator as shown in the example, and the configuration of the regulator please refer to Documentation/devicetree/bindings/regulator/.

It also needs special attention, if it is a PCIe3.0 controller, generally an external 100M crystal chip is needed, and the power supply of this crystal chip is shared with the 3V3 of the PCIe peripheral in the hardware design. So after configuring this item, in addition to confirming the 3V3 power supply of the peripheral, it is also necessary to confirm whether the clock output of the external crystal chip is normal. Generally, an external crystal chip needs a stable period to output the clock. Therefore, please strictly refer to the minimum value specified in the manual of the clock chip, and on the basis of having a test margin, specify the value of the startup-delay-us attribute in the power supply node. In addition, for hardware designs that discharge slowly after power off, specify the value of the off-on-delay-us attribute in the power supply node to ensure that the power on and off operations are sufficient. Taking rk3568 as an example, a detailed example can be referred to the vcc3v3_pcie node in the rk3568-evb1-ddr4-v10.dtsi file.

7. `phys`

Optional Configuration Item: Used to configure the phandle reference of the phy used by the controller, some controllers can route to multiple phys (such as RK3588's pcie2x1l0 and pcie2x1l1), it should be noted that different phy reference methods may vary, comboPHY needs to specify the working mode of the phy at the same time, as follows:

```
    phys = <&pcie30phy>;
    phys = <&combphy1_ps PHY_TYPE_PCIE>;
```

8. `rockchip,bifurcation;`

Optional Configuration Item: This is a unique configuration for the **RK3568 chip**. It can split the 2 lanes of pcie3x2 into two 1-lane controllers for use. The specific configuration method is to enable the pcie3x1 and pcie3x2 controller nodes and pcie30phy in the dts, and add the rockchip,bifurcation property to both the pcie3x2 and pcie3x1 nodes. For reference, see rk3568-evb6-ddr3-v10.dtsi. Otherwise, by default, the pcie3x1 controller cannot be used.

At this time, lane0 is used by the pcie3x2 controller, and lane1 is used by the pcie3x1 controller, and the hardware layout should strictly follow our company's schematic. Also note that under this mode, the two 1-lane controllers must work simultaneously in RC mode.

It also needs special attention, after PCIe 3.0 is split into two single lanes and connected to two different peripherals, since the crystal and its power supply are controlled by the same route. At this time, do not configure vpcie3v3-supply to one of the controllers, otherwise, it will cause the controller that has obtained the 3v3 voltage operation rights to interfere with the normal initialization of the peripheral connected to the other controller. At this time, the regulator corresponding to vpcie3v3-supply should be configured as regulator-boot-on and regulator-always-on.

9. `prsnt-gpios = <&gpio4 15 GPIO_ACTIVE_LOW>;`

Optional Configuration Item: Used to drive the recognition of whether there is a peripheral and related peripheral circuit, if an effective level is detected, the device detection process is skipped. According to the PCIe Electrical Characteristics Protocol Document, this gpio indicates that a device is connected when it is low. If the board design is the opposite, it can be modified to GPIO_ACTIVE_HIGH to indicate that a high level indicates a device is connected. This signal is used to support the same board type with PCIe3 and without PCIe3 with the same set of software, to avoid the system exception of rcu stall during the initialization of the pcie controller;

10. `rockchip,perst-inactive-ms = <500>;`

Optional Configuration Item: Used to configure the reset time of the device #PERST reset signal, in milliseconds. According to the PCIe Express Card Electromechanical Spec, the minimum requirement for the downstream device power to stabilize and release #PERST is 100ms, if this item is not configured, the RK driver defaults to a configuration of 200ms. If it still does not meet the working requirements of the peripheral, it can be adjusted as appropriate, based on actual measurement.

11. `rockchip,s2r-perst-inactive-ms = <1>;`

Optional Configuration Item: Used to configure the reset time of the device #PERST reset signal during sleep wake-up, in milliseconds. If not configured, its value is equal to the configuration of `rockchip,perst-inactive-ms`. If the peripheral does not power off during sleep, taking Wi-Fi as an example, then #PERST is in a reset state during sleep, so the reset time of #PERST can be shortened during wake-up, and it can even be configured to 0.

12. `rockchip,wait-for-link-ms = <1>;`

Optional Configuration Item: Used to configure the waiting time after the release of the device #PERST reset signal, in milliseconds. This configuration is used for some peripherals that require a longer time for internal initialization to prevent the system from timing out due to the long internal initialization of the peripherals. Currently, the common ones that require this configuration are FPGA and some AI computing cards.

13. `supports-clkreq;`

Optional Configuration Item: Only valid in RC mode, please confirm that the CLKREQN pinctrl iomux has been configured as function io, if this attribute exists, it specifies the existence of a signal route from the root port to the downstream device CLKREQN, and the host bridge driver program can be programmed according to the existence of the CLKREQN signal, for example, if there is no CLKREQN signal, the root port is set as not supporting PM L1 Substates.

14. `rockchip,lpbk-master`

Special Debug Configuration: This configuration is for loopback signal testing, using the PCIe controller to construct a simulated loopback master environment, allowing the device under test to enter the slave model, do not configure if it is not a simulated verification laboratory RX loop requirement. Also note that Gen3 controllers may need to configure compliance mode to loopback slave mode. If the reader does not understand what loopback testing is, it means this is not the configuration you are looking for, do not ask questions about this configuration.

15. `rockchip,compliance-mode`

Special Debug Configuration: This configuration is for compliance signal testing, forcing the PCIe controller to enter compliance test mode or when using the SMA fixture to enter the test mode without power off. This is an array containing two configurations, the first configuration of the array indicates the test mode, and the second configuration indicates the preset value under the aforementioned configuration. If using the SMA fixture for testing, it is recommended to configure `rockchip,compliance-mode=<0 0>;` ; if testing soldered devices, it is necessary to fix the configuration mode and preset value, `rockchip,compliance-mode=<mode preset>;` . The mode should be configured as 1, 2, or 3 according to the need, representing 2.5GT, 5.0GT, and 8GT signals respectively. Only in 5GT and 8GT modes, the preset's valid values are 0 to 10, representing different protocol pre-emphasis configurations such as P0 to P10, refer to the "About PCIe TX Emphasis Preset Value table" section in the appendix.

16. `rockchip,keep-power-in-suspend`

Optional Configuration Item: Only valid in RC mode, used to achieve not turning off the power of the peripheral and resetting it during sleep. Allows the peripheral to work offline after the system sleeps. This mode also requires the pcie node to reference `vpcie3v3-supply`.

2.6.2 comboPHY dts Configuration

Note:

- The following configurations are not applicable to the combphy node of RK1808.
- The combphy node number indicates the Mux relationship, and the suffix indicates the multiplexing relationship, with p, s, u, q representing PCIe, SATA, USB, and QSGMII respectively.

1. `rockchip,ext-refclk`

Special Debug Configuration: Please note that this configuration is specifically for combophy. By default, combphy uses the SoC internal clock scheme. For example, in RK356X, you can refer to the rk3568.dtsi node, which uses a 24MHz clock source by default. In addition to the 24MHz clock source, it also supports 25M and 100M, and you only need to adjust the assigned-clock-rates = <24000000> value to the desired frequency. The internal clock source scheme is the most cost-effective, so it is used as the SDK default scheme. However, combphy still reserves the option for an external crystal chip clock source input. If you really need to use an external clock crystal chip to provide a clock scheme, please add rockchip,ext-refclk in the combphy node used by the PCIe controller in the board-level dts, and pay attention to adding assigned-clock-rates = in the node to specify the frequency of the external clock chip input, still only supporting 24M, 25M, 100M three gears.

2. `rockchip,enable-ssc`

Special Debug Configuration: Please note that this configuration is specifically for the combphy node used by PCIe. By default, the combophy output clock does not enable spread spectrum clocking. If users need to avoid some EMI issues, they can try adding this configuration item to the corresponding combphy node to enable SSC.

2.6.3 PCIe30 PHY DTS Configuration

1. `rockchip,pcie30-phymode`

Optional Configuration Item: This configuration is for the combined usage mode of PCIe30 PHY, which needs to be properly set up, with the default being 4Lane shared. For detailed optional content, refer to `include/dt-bindings/phy/phy-snps-pcie3.h`:

```
/*
 * pcie30_phy_mode[2:0]
 * bit2: aggregation
 * bit1: bifurcation for port 1
 * bit0: bifurcation for port 0
 */
#define PHY_MODE_PCIE_AGGREGATION 4  /* PCIe3x4 */
#define PHY_MODE_PCIE_NANBNB 0   /* P1:PCIe3x2  +  P0:PCIe3x2 */
#define PHY_MODE_PCIE_NANBBI 1   /* P1:PCIe3x2  +  P0:PCIe3x1*2 */
#define PHY_MODE_PCIE_NABINB 2   /* P1:PCIe3x1*2 + P0:PCIe3x2 */
#define PHY_MODE_PCIE_NABIBI 3   /* P1:PCIe3x1*2 + P0:PCIe3x1*2 */
```

2.7 Fill DTS Based on Schematic Diagram

2.7.1 Low-Speed IO Description

The chip signal connections of the PCIe module, in addition to the data lines and reference clock differential pairs, may also include the following low-speed IOs:

Low-Speed IO Name	Function	Description
PERSTN	GPIO Output	Mandatory, configure dts "reset-gpios" item
WAKEN	GPIO (PMU Domain) Input	Optional, function driver registers corresponding GPIO interrupt and wake-up source, not handled by the PCIe controller driver
PWREN	GPIO Output	Optional, configure dts "vpcie3v3-supply" item
CLKREQN	FUNCTION	Optional, used when supporting L1SS, configure dts "supports-clkreq" item
PRSNTN	GPIO Input	Optional, configure dts "prsnt-gpios" item

Explanation:

- CLKREQN uses the function feature of PCIe, this signal is only required for L1SS functionality, otherwise it can be left unconnected, and the corresponding dts attribute must be added when used;
- For GPIO pins, do not configure them as PCIe functions in pinctrl, refer to the corresponding dts configuration for specific usage;
- The PERSTN signal is a mandatory signal required by the protocol, other signals are configured according to the actual project requirements;
- For pin explanations in PCIe EP mode, refer to the relevant sections of the document "Rockchip_Developer_Guide_PCIE_EP_Standard_Card_CN".

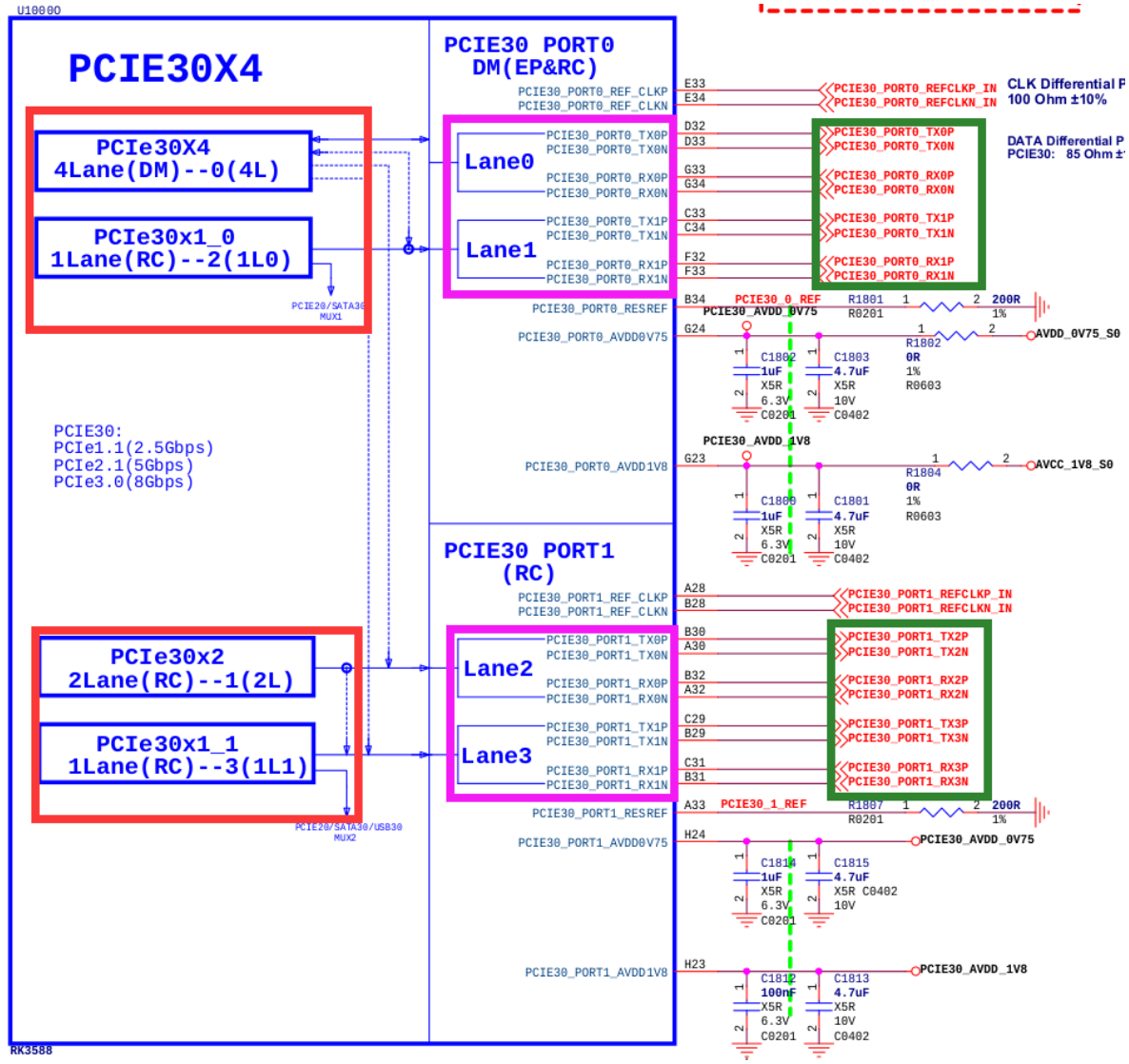
2.7.2 DTS Configuration Method

Schematic diagrams describe hardware from the perspective of IO signals, which are strongly related to the PHY index. It was mentioned earlier that the controller and PHY index of RK3588 may not be consistent, so special attention is needed when looking at the schematic diagrams. Here are some suggestions for filling in and examples to illustrate how to correspond the PHY and controller from the schematic diagram to the DTS node.

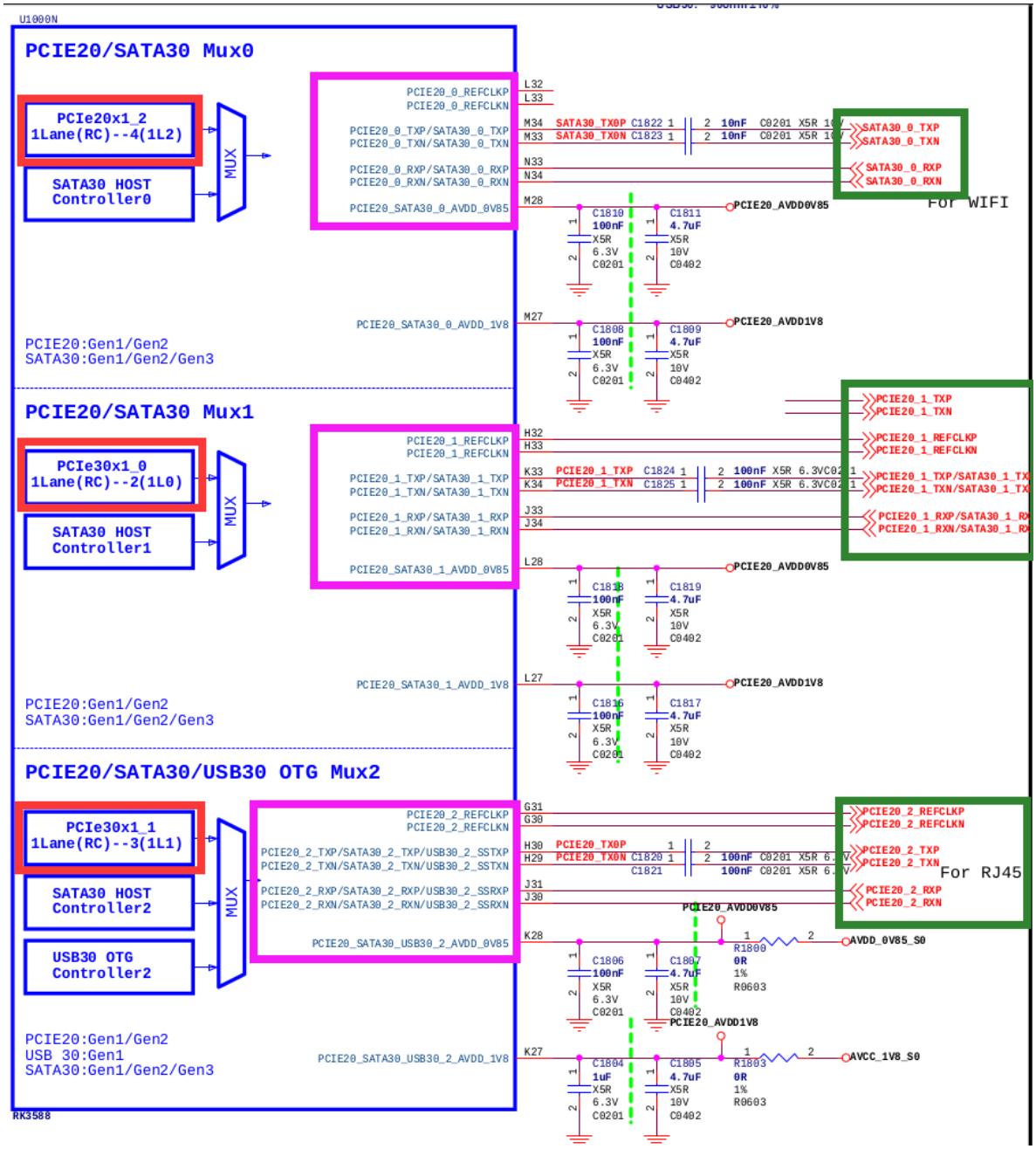
Recommended steps for filling in DTS based on hardware schematic diagrams:

1. Confirm with hardware engineers how many PCIe devices are used and how the chip's multiple PCIe interfaces are allocated;
2. Identify in the schematic diagram which PHY output corresponds to the PCIe data lines used by a certain device;
3. Determine which controller and PHY the current device uses, and enable them in DTS;
4. Ensure the "phy" attribute and mode of the controller used by the current PCIe interface in DTS are correctly selected, such as the pcie2x1ln controller needing to choose comboPHY and specify PHY_TYPE_PCIE;
5. ComboPHY may be shared by multiple controllers such as SATA, USB, RGMII, etc., please confirm that the corresponding other controllers are disabled in DTS;
6. Determine if the current PHY has multiple working modes and if the configuration is correct, such as the different split combinations of pcie30phy requiring correct mode configuration;
7. Determine which GPIO the "PERSTn" signal used by the current PCIe interface is, and correctly configure it to the controller DTS node;
8. Determine which GPIO controls the "PWREN" signal used by the current PCIe interface, and correctly configure it to the controller DTS node (this configuration can also be placed in the DTS of on-board peripherals);
9. Configure other hardware required for peripheral operation;

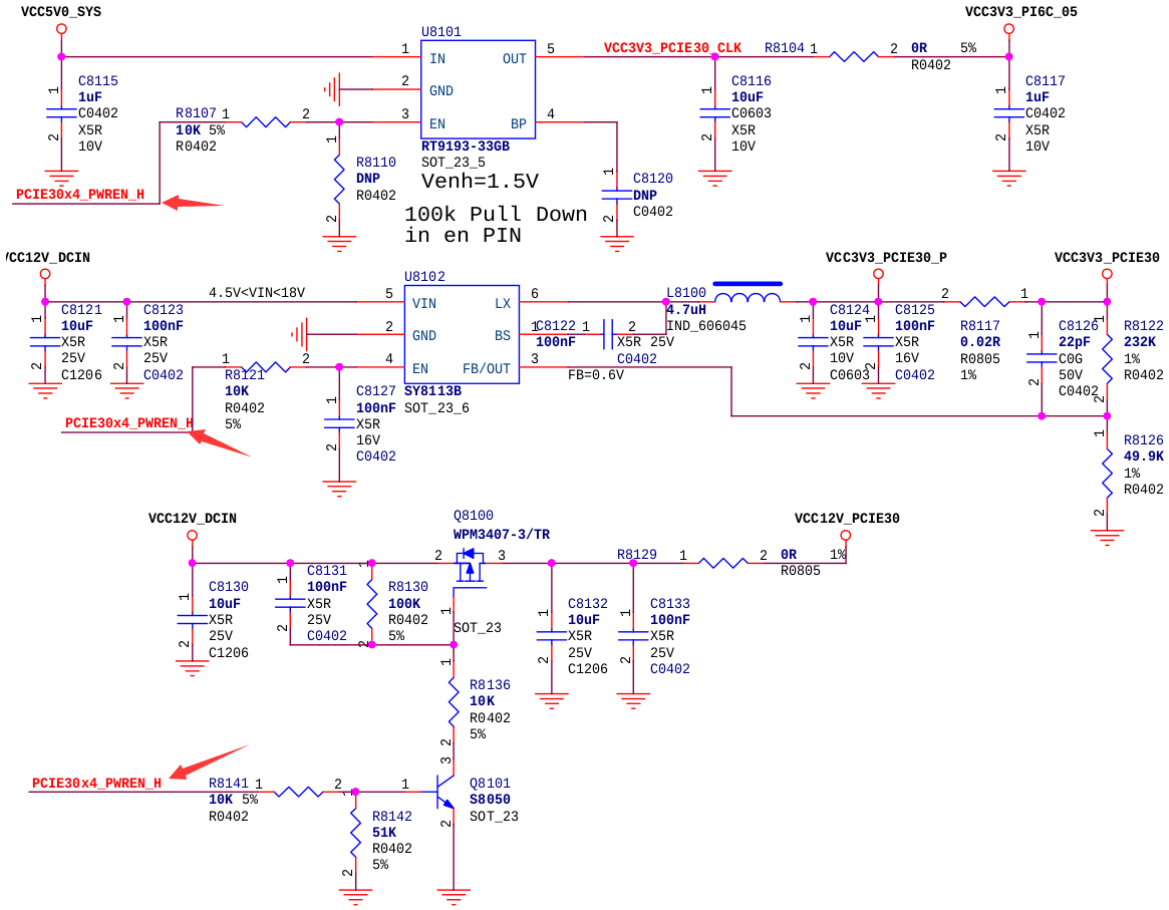
The following figure is the RK3588 pcie30phy and its possible controllers, with the red box representing the controller, the pink box representing the PHY signal, and the green box representing the peripheral signal; the actual controller used can be confirmed by the connection of peripheral signals, or checked with hardware engineers to understand if it is correct. The figure is from RK3588 evb1, with a PCIe3.0 x4 slot connected, so the controller used is PCIe30X4 (DTS named pcie3x4), and the other controllers are not used with this PHY.



The following figure is the RK3588 comboPHY and its possible controllers, with the red box representing the controller, the pink box representing the PHY signal, and the green box representing the peripheral signal; the actual controller used can be confirmed by the connection of peripheral signals, or checked with hardware engineers to understand if it is correct. In this figure, Mux0's PHY (combphy0_ps) operates in SATA mode, not PCIe; Mux1's PHY (combphy1_ps) works with PCIe30x1_0 (DTS named pcie2x1l0) and may operate in PCIe mode, which needs to be determined by the actual device connected; Mux2's PHY (combphy2_psu) works with PCIe30x1_1 (DTS named pcie2x1l1) in PCIe mode for connecting a PCIe network card.



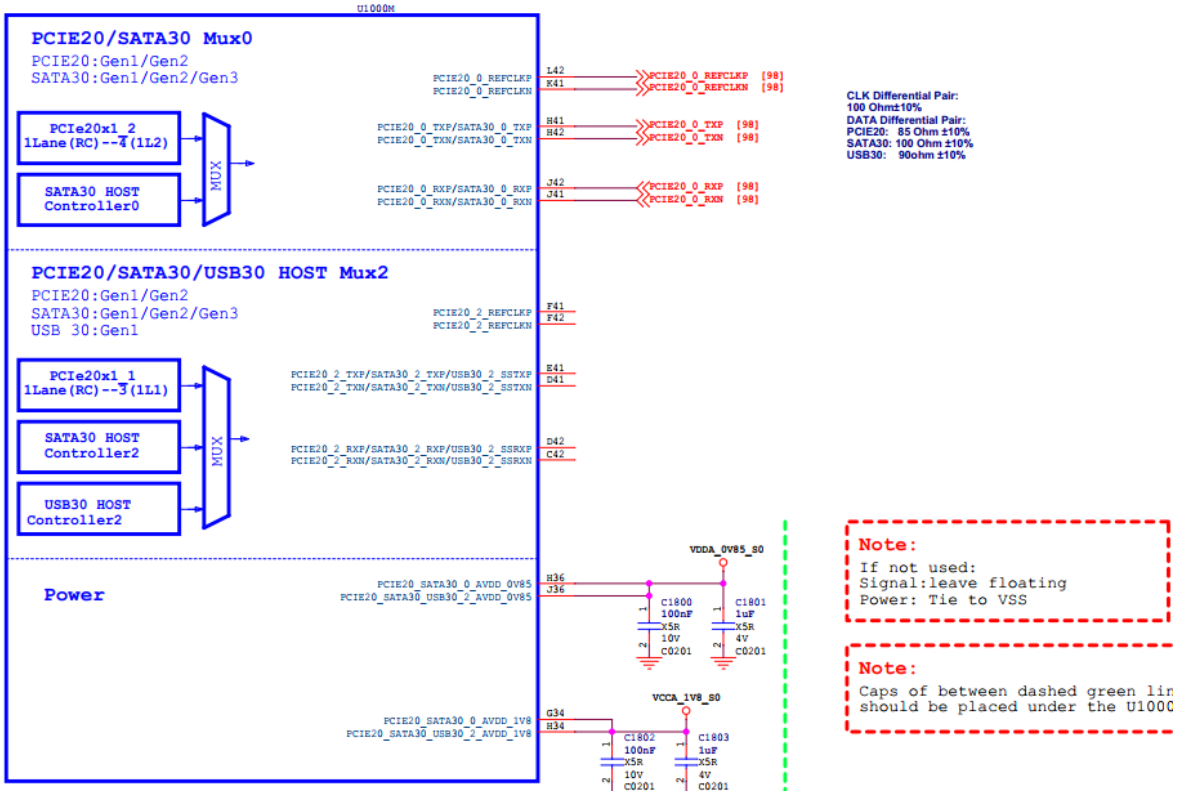
The following figure shows the PCIe3.0 interface power supply of RK3588 evb1, which can be located by the signal PCIE30X4_PWREN_H corresponding to the RK3588 GPIO, and then filled into the pcie3x4 controller DTS.



2.8 Wi-Fi Module Device Tree Writing Example

Due to the Wi-Fi module generally connecting to the PCIe2.0 interface, its combphy multiplexing relationships are complex, and its power usage, sleep modes, and reset requirements are distinctly different from other devices. Here is a device tree writing example for reference.

1. Review the schematic to understand the combphy used by the Wi-Fi module. The combphy node number indicates the Mux relationship, and the suffix indicates the multiplexing relationship, with p, s, u, q representing PCIe, SATA, USB, and QSGMII, respectively. Taking this diagram as an example, Wi-Fi is connected to the PCIe and SATA multiplexed PCIE20/SATA30 Mux0, so the configured should be the combphy0_ps node.



2. Search the dts file to ensure that other controller nodes using this combphy function are disabled to prevent signal interference.
3. Move the wifi_reg_on signal from the wireless_wlan node to the PCIe 3.3v power control node.
4. If Wi-Fi needs to achieve L1.x power consumption mode, please refer to the "RC mode PM L1 Substates Support" section.
5. If Wi-Fi needs to implement wireless wake-up functionality, ensure that the wifi_reg_on pin remains high during sleep, and the wifi_host_wake pin is connected to the non-power-off PMU IO for generating interrupt wake-up to the main controller. For detailed hardware connections and software modifications, please refer to the relevant sections in our company's SDK release documents, "Rockchip_Developer_Guide_Linux_WIFI_BT_CN.pdf".

```
+ vcc3v3_pcie20_wifi: vcc3v3-pcie20-wifi {
+      compatible = "regulator-fixed";
+      regulator-name = "vcc3v3_pcie20_wifi";
+      regulator-min-microvolt = <3300000>;
+      regulator-max-microvolt = <3300000>;
+      enable-active-high;
+     /*
+      * wifi_reg_on is enabled after vbat and vddio are stable, pulled high
before reset-gpios, so
+      * it is placed in the PCIe power node to meet the module timing, referring
to wifi_poweren_gpio.
+      */
+      pinctrl-0 = <&wifi_poweren_gpio>;
+      startup-delay-us = <5000>;
+      vin-supply = <&vcc12v_dcin>;
+    };

     wireless_wlan: wireless-wlan {
         compatible = "wlan-platdata";
         wifi_chip_type = "ap6275p";
         pinctrl-names = "default";
         pinctrl-0 = <&wifi_host_wake_irq>;
         WIFI,host_wake_irq = <&gpio0 RK_PA0 GPIO_ACTIVE_HIGH>;
```

```
+       /* Note: The wifi_reg_on pin also needs to be configured here for the
WiFi driver to control */
+       WIFI,poweren_gpio = <&gpio0 RK_PC7 GPIO_ACTIVE_HIGH>;
        status = "okay";
    };
+
+&sata0 {
+    status = "disabled" /* sata0 and pcie2x1l2 share combphy0_ps, ensure it is
disabled */
+}
+
+&combphy0_ps {
+    status = "okay"; /* Ensure phy is enabled */
+};
+
+&pcie2x1l2 {
+    reset-gpios = <&gpio3 RK_PD1 GPIO_ACTIVE_HIGH>;
+    rockchip,skip-scan-in-resume;
+    rockchip,perst-inactive-ms = <500>; /* Refer to the Wi-Fi module manual for
the required #PERST reset time */
+    vpcie3v3-supply = <&vcc3v3_pcie20_wifi>;
+    status = "okay";
+};

&pinctrl {
     wireless-wlan {
         wifi_poweren_gpio: wifi-poweren-gpio {
+           //PCIE REG ON: Be sure to configure it as pull-up
+           rockchip,pins = <0 RK_PC7 RK_FUNC_GPIO &pcfg_pull_up>;
         };
     };
};
```

3. menuconfig Configuration

1. Ensure the following configurations are enabled to correctly use PCIe-related functions:

```
CONFIG_PCI=y
CONFIG_PCI_DOMAINS=y
CONFIG_PCI_DOMAINS_GENERIC=y
CONFIG_PCI_SYSCALL=y
CONFIG_PCI_BUS_ADDR_T_64BIT=y
CONFIG_PCI_MSI=y
CONFIG_PCI_MSI_IRQ_DOMAIN=y
CONFIG_PHY_ROCKCHIP_SNPS_PCIE3=y
CONFIG_PHY_ROCKCHIP_NANENG_COMBO_PHY=y
CONFIG_PCIE_DW=y
CONFIG_PCIE_DW_HOST=y
CONFIG_PCIE_DW_ROCKCHIP=y
CONFIG_PCIEPORTBUS=y
CONFIG_PCIE_PME=y
CONFIG_GENERIC_MSI_IRQ=y
CONFIG_GENERIC_MSI_IRQ_DOMAIN=y
```

```
CONFIG_IRQ_DOMAIN=y
CONFIG_IRQ_DOMAIN_HIERARCHY=y
```

2. Enable NVMe devices (SSDs based on the PCIe interface), PCIe-to-AHCI devices (SATA), and PCIe-to-USB devices (XHCI) are all enabled in the default config, please confirm. For other devices such as Ethernet cards, WiFi, etc., please confirm the relevant config settings yourself.

```
CONFIG_BLK_DEV_NVME=y
CONFIG_SATA_PMP=y
CONFIG_SATA_AHCI=y
CONFIG_SATA_AHCI_PLATFORM=y
CONFIG_ATA_SFF=y
CONFIG_ATA=y
CONFIG_USB_XHCI_PCI=y
CONFIG_USB_XHCI_HCD=y
```

Special Note: The default kernel only supports PCIe-to-SATA devices listed in drivers/ata/ahci.c. For devices beyond this list, please seek support from the original manufacturer or distributor.

4. Standard EP Function Development

The PCIe controller of RK series chips supports EP mode, allowing the chips to be developed into standard PCIe EP products. For the implementation of EP functionality, please refer to the document **"Rockchip_Developer_Guide_PCIE_EP_Standard_Card"**.

5. RC mode PM L1 Substates Support

When it is confirmed that the RK main controller and the connected peripherals support PCIe PM L1 Substates, further optimization of power consumption can be achieved by enabling the PM L1 Substates feature.

Further emphasis is placed on the fact that if there are devices among the target peripherals that do not support PM L1 Substates, especially if the motherboard design involves slot-connected non-fixed devices, do not enable the PM L1 Substates feature, as some devices may not function properly.

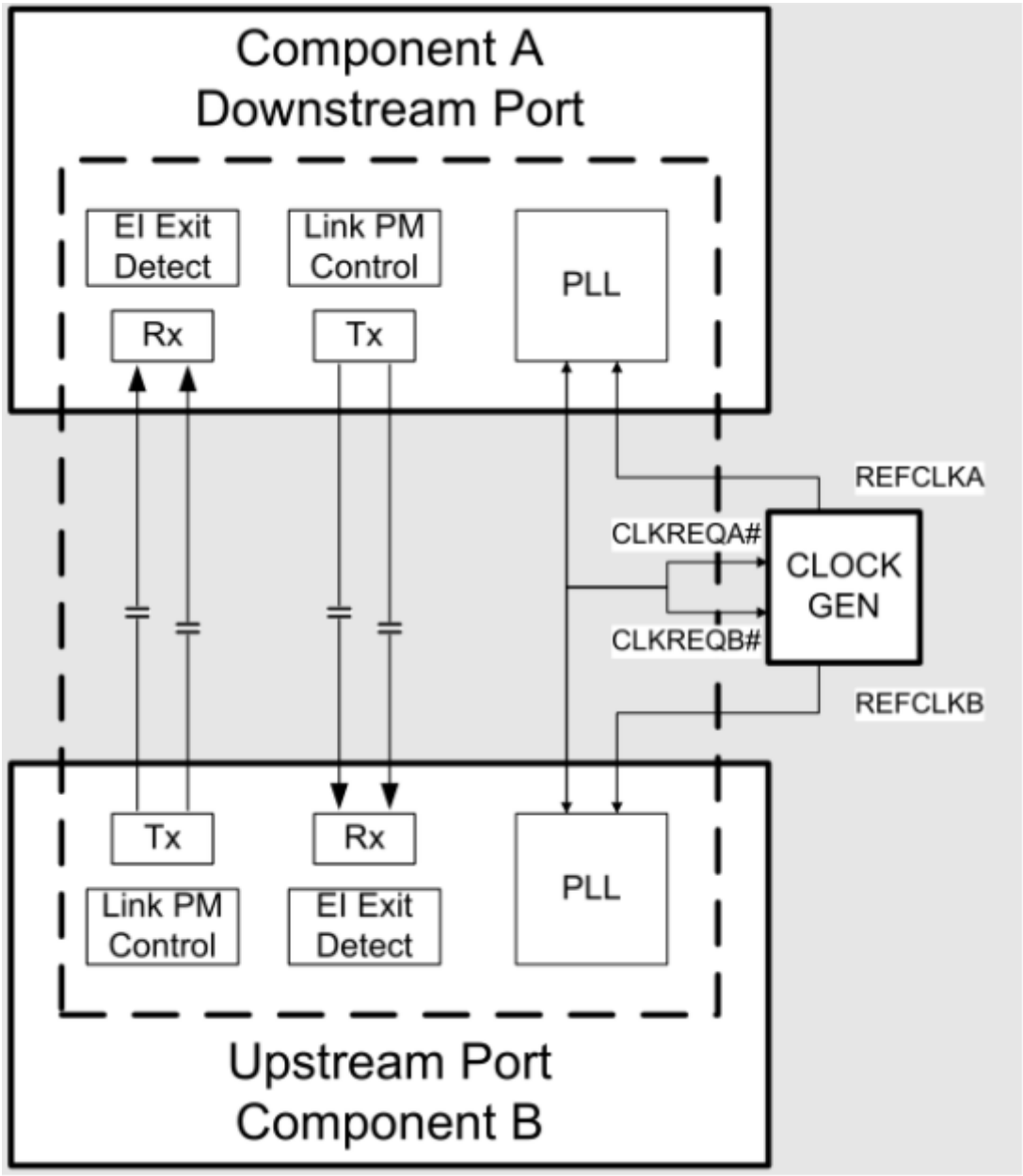
Notice

- The RK PCIe L1SS feature has a TL1O_REFCLK_ON timing of less than 10us, which is not adjustable and is theoretically compatible with most peripherals. However, some peripherals have strict requirements for this specification, so it is recommended to further confirm the specification constraints with the original equipment manufacturers of the relevant peripherals.
- The default L1 substate exit time `Tpower_on` for the RK PCIe is 10us, which meets its own timing requirements. However, the peripheral device's `Tpower_on` might not be sufficient to ensure its proper operational timing. If this requirement is not met, link instability may occur. It is recommended to further confirm this parameter with the respective peripheral vendor.

Hardware Circuit Design Supporting PM L1 Substates:

- Internal reference clock scheme: The CLKREQN signal of the Root Complex (RC) and the CLKREQN signal of the Endpoint (EP) are interconnected at both ends.

- External reference clock scheme: The CLKREQN signal of the Root Complex (RC) is connected to the enable pin (OE) of the CLOCK_GEN. If the enable pin (OE) of the CLOCK_GEN is active high, an inverter should be added.



The system does not enable PM L1 Substates support by default.:

- The "supports-clkreq;" property is not added in the dts.
- The kernel macro configuration CONFIG_PCIEASPM_POWER_SUPERSAVE=n.

System Enable PM L1 Substates Support

- Confirm that both RC/EP support PM L1 Substates, with the RC support status available in the "Chip Resource Introduction" section under the ASPM column.
- Hardware configuration of CLKREQN signal: Ensure compliance with "Hardware Circuit Design Supporting PM L1 Substates."
- Software configuration of CLKREQN signal: Set CLKREQN iomux to function io.
- Software configuration of the controller node adds the "supports-clkreq;" property, for details refer to "DTS Configurable Item 11" explanation.
- Enable PM L1 Substates support:
    - Method 1: Kernel macro configuration CONFIG_PCIEASPM_POWER_SUPERSAVE=y

- Method 2: Some peripherals, such as WIFI, support enabling PM L1 Substates in the driver.

Explanation:

- If the above conditions are not strictly met and an attempt is made to enable the kernel macro configuration, the PCIe link may enter an abnormal state and fail to wake up.
- Please confirm that the kernel source code is up-to-date and includes the support patch for L1SS: commit e18dfa93 PCI: rockchip: dw: Support PM L1 clock removing.

6. GPIO-Based Detection Mechanism for Plug and Unplug Events

6.1 Hardware Requirements

1. The PRSNTN_1 of the PCIe slot must be connected to any GPIO of the main controller to serve as a detection pin.
2. The power supply of the PCIe device must be controllable by software for power on and off.

6.2 Software Requirements

1. Submissions must include at least the following, if not, please contact the business for patches:

```
commit 4de1a0c19e0f9804ba22e7f5e544fea317913957
Author: Shawn Lin <shawn.lin@rockchip.com>
Date:   Tue Mar 12 16:38:46 2024 +0800

    PCI: rockchip: dw: Add gpio based hotplug
    Change-Id: I49c57755d11cc43bbf7cf9eb23542f5e1e11aaa3

commit 86f3010d7f523c9f5a2e88d9f8f1871ed89da098
Author: Vidya Sagar <vidyas@nvidia.com>
Date:   Sat Oct 1 00:57:45 2022 +0530

    FROMLIST: PCI/hotplug: Add GPIO PCIe hotplug driver
    Change-Id: Iafa798ee4d98f195f5d33d80120da0c569132548
```

2. The kernel must confirm that the following configurations are enabled:

```
CONFIG_HOTPLUG_PCI=y
CONFIG_HOTPLUG_PCI_GPIO=y
```

3. DTS configuration reference as follows

```
&pcie0 {
  reset-gpios = <&gpio4 RK_PC7 GPIO_ACTIVE_HIGH>;
  vpcie3v3-supply = <&vcc3v3_pcie0>;
  hotplug-gpios = <&gpio4 RK_PC4 IRQ_TYPE_EDGE_BOTH>;
  pinctrl-names = "default";
  pinctrl-0 = <&hot_plug0>;
```

```
  status = "okay";
};

&pinctrl {
  pcie {
      hot_plug0: hot-plug0 {
          rockchip,pins = <4 RK_PC4 RK_FUNC_GPIO &pcfg_pull_up>;
      };
  };
};
```

6.3 Usage Restrictions

1. Hot-plugging of PCIe devices while powered can easily damage the device and the controller. The unloading and power-off procedures after device removal require some time, so rapid hot-plugging is prohibited. Wait until the following removal log appears before reinserting:

```
[   35.680289][  T134] pcieport 0000:00:00.0: Hot-UnPlug Event
[   35.680361][  T134] pcieport 0000:00:00.0: Power Status = 1
[   35.827183][  T134] rk-pcie 2a200000.pcie: rk_pcie_slot_disable
[   35.827303][  T134] pcieport 0000:00:00.0: Hot-UnPlug Event
[   35.827323][  T134] pcieport 0000:00:00.0: Power Status = 0
[   35.827334][  T134] pcieport 0000:00:00.0: Device is already removed
```

2. To ensure data integrity and system stability, it is necessary to ensure that the system stops accessing the device to be removed.
3. It is not possible to support the individual hot-plugging of downstream devices of the switch. If there is such a need, first confirm that the switch supports individual hot-plugging of downstream devices, then refer to the "How to rescan or replace a single downstream device online?" section in the common application issues for rescan processing, which can achieve the same effect.
4. It is not supported to detect the insertion or removal of devices in a sleep or standby state.

7. Kernel DMATEST

Before development, please confirm whether the target PCIe controller supports DMA transfer, refer to the "Chip Resource Introduction" section for details.

RK PCIe DMA provides a test mechanism based on the kernel module_para mechanism, similar to the Linux dmatest framework, which can be further developed for kernel-level PCIe DMA applications.

Kernel Version Requirements

Include at least the following commit, if not, please contact the business for patches:

```
commit a7c40cb119703e566d9d5befb8c1a7b0533dd7b7
Author: Jon Lin <jon.lin@rock-chips.com>
Date:   Tue Jan 17 17:46:48 2023 +0800

    PCI: rockchip: dw-dmatest: Suppport rc dma

    1.Set rc dma as default
    2.Changet to ep dma by sending command:
            echo 0 > ./sys/module/pcie_dw_dmatest/parameters/is_rc

    Change-Id: I9b16c328c08f220772e487c7c796b8898d74ae10
    Signed-off-by: Jon Lin <jon.lin@rock-chips.com>
```

Test Macro Configuration

```
CONFIG_PCIE_DW_DMATEST=y
CONFIG_ROCKCHIP_PCIE_DMA_OBJ=n
```

Setting Up the Environment

PCIe Interconnection Model:

- Customers set up RK RC - FPGA EP environment on their own
- RK local testing setup is RK RC (RK dmatest configuration) - RK EP (RK chip interconnection configuration)

Notes:

- MPS configuration is 256B
- Both RC and EP require reserved memory, it is recommended to reserve 64MB of test memory at 0x3c000000 (the default configuration address for testing)
- It is recommended to disable other unrelated PCIe controllers

Testing

Refer to the pcie-dw-misc-dmatest.c source code for details.

```
static int size = 0x20;
module_param(size, int, 0644);
MODULE_PARM_DESC(size, "each packet size in bytes");

static unsigned int cycles_count = 1;
module_param(cycles_count, uint, 0644);
MODULE_PARM_DESC(cycles_count, "how many erase cycles to do (default 1)");

static bool irq;
module_param(irq, bool, 0644);
MODULE_PARM_DESC(irq, "Using irq? (default: false)");

static unsigned int chn_en = 1;
module_param(chn_en, uint, 0644);
MODULE_PARM_DESC(chn_en, "Each bits for one dma channel, up to 2 channels,
(default enable chanel 0)");

static unsigned int rw_test = 3;
module_param(rw_test, uint, 0644);
MODULE_PARM_DESC(rw_test, "Read/Write test, 1-read 2-write 3-both(default 3)");
```

```
static unsigned int bus_addr = 0x3c000000;
module_param(bus_addr, uint, 0644);
MODULE_PARM_DESC(bus_addr, "Dmatest chn0 bus_addr(remote), chn1 add offset
0x100000, (default 0x3c000000)");

static unsigned int local_addr = 0x3c000000;
module_param(local_addr, uint, 0644);
MODULE_PARM_DESC(local_addr, "Dmatest chn0 local_addr(local), chn1 add offset
0x100000, (default 0x3c000000)");

static unsigned int test_dev;
module_param(test_dev, uint, 0644);
MODULE_PARM_DESC(test_dev, "Choose dma_obj device,(default 0)");

static bool is_rc = true;
module_param_named(is_rc, is_rc, bool, 0644);
MODULE_PARM_DESC(is_rc, "Test port is rc(default true)");
```

Example Reference 1: RC device, channel 0, write data, data granularity 1MB, loop count 1000, local address 0x3c000000, remote address 0x3c000000:

```
echo 0 > ./sys/module/pcie_dw_dmatest/parameters/test_dev
echo 1 > ./sys/module/pcie_dw_dmatest/parameters/is_rc
echo 1 > ./sys/module/pcie_dw_dmatest/parameters/chn_en
echo 1 > ./sys/module/pcie_dw_dmatest/parameters/rw_test
echo 0x100000 > ./sys/module/pcie_dw_dmatest/parameters/size
echo 1000 > ./sys/module/pcie_dw_dmatest/parameters/cycles_count
echo 0x3c000000 > ./sys/module/pcie_dw_dmatest/parameters/local_addr
echo 0x3c000000 > ./sys/module/pcie_dw_dmatest/parameters/bus_addr
echo run > ./sys/module/pcie_dw_dmatest/parameters/dmatest
```

Example Reference 2: EP device, channel 0/1 (dual-threaded operation), read/write data, data granularity 8KB, loop count 10000, local address 0x3c000000, remote address 0x3c000000:

```
echo 0 > ./sys/module/pcie_dw_dmatest/parameters/test_dev
echo 0 > ./sys/module/pcie_dw_dmatest/parameters/is_rc
echo 3 > ./sys/module/pcie_dw_dmatest/parameters/chn_en
echo 3 > ./sys/module/pcie_dw_dmatest/parameters/rw_test
echo 0x2000 > ./sys/module/pcie_dw_dmatest/parameters/size
echo 10000 > ./sys/module/pcie_dw_dmatest/parameters/cycles_count
echo 0x3c000000 > ./sys/module/pcie_dw_dmatest/parameters/local_addr
echo 0x3c000000 > ./sys/module/pcie_dw_dmatest/parameters/bus_addr
echo run > ./sys/module/pcie_dw_dmatest/parameters/dmatest
```

8. Kernel Stability Statistics

PCIe devices may exhibit anomalies under long-term operation, deviating from expected behavior. To facilitate analysis, you can retrieve information from the debugfs node. To enable this feature, ensure that the commit (`pcie: rockchip: dw: Add debugfs support`) is included; otherwise, you can obtain the `0001-pcie-rockchip-dw-Add-debugfs-support.patch` from the Redmine system as shown in the appendix.

Usage Method:

1. Identify the controller address node of the problematic device, which can be found in the boot enumeration log or directly from the dtsi.
2. Taking `fe16000.pcie` as an example, navigate to the `/sys/kernel/debug/fe160000.pcie` directory.
3. `echo disable > err_event` to disable all event statistics functions.
4. `echo clear > err_event` to clear all event statistics counters.
5. `echo enable > err_event` to enable all event statistics functions.
6. Begin device aging to reproduce your exceptional case. After reproduction, execute `cat dumpfifo` and `cat err_event`.
7. Compare the exported information with the Debugfs Export Information Parsing Table in the appendix of this document to roughly identify the issue.

9. Kernel AER support and Injection Test Support

The AER (Advanced Error Reporting) function is a standard component provided by the Linux framework. It reports underlying communication errors to various user-space daemons and establishes fault-tolerant handling mechanisms. In complex business environments, AER injection testing can simulate various AER events in advance to improve exception handling procedures.

1. Enable kernel AER and injection support to create the `/dev/aer_inject` node in the system:

```
CONFIG_PCIEAER=y
CONFIG_PCIEAER_INJECT=y
```

2. Configure kernel's cmdline

```
#Use RK3588-EVB Linux for example,  add pcie_pme=nomsi
--- a/arch/arm64/boot/dts/rockchip/rk3588-linux.dtsi
+++ b/arch/arm64/boot/dts/rockchip/rk3588-linux.dtsi
@@ -12,7 +12,7 @@ aliases {
        };

        chosen: chosen {
-               bootargs = "earlycon=uart8250,mmio32,0xfeb50000 console=ttyFIQ0
irqchip.gicv3_pseudo_nmi=0 root=PARTUUID=614e0000-0000 rw rootwait
rcupdate.rcu_expedited=1 rcu_nocbs=all";
+               bootargs = "earlycon=uart8250,mmio32,0xfeb50000 console=ttyFIQ0
irqchip.gicv3_pseudo_nmi=0 root=PARTUUID=614e0000-0000 rw rootwait
rcupdate.rcu_expedited=1 rcu_nocbs=all pcie_pme=nomsi";
        };
```

3. Make sure we get it right and register aer irq service

```
# check the cmdline for sure we set pcie_pme=nomsi
cat /proc/cmdline
# check the irq
root@debian:/userdata# cat /proc/interrupts | grep aerdrv
 60:      0      0      0      0      0      0      0      0     INTx   0 Edge
PCIe PME, aerdrv, PCIe bwctrl
 63:      0      0      0      1      0      0      0      0     INTx   0 Edge
PCIe PME, aerdrv
110:      0      0      0      0      0      0      0      0     INTx   0 Edge
PCIe PME, aerdrv
```

4. Obtain the AER injection tool `aer-inject` from the "Development Resource Acquisition Address" mentioned in the appendix of this document.
5. Query the address of the target PCIe device for testing:

```
# Use lspci to obtain the PCIe device address. Example with network card
01:00.0:
00:00.0 PCI bridge: Rockchip Electronics Co., Ltd Device 3576 (rev 01)
01:00.0 Network controller: Broadcom Inc. and subsidiaries Device 449d (rev 02)
```

4. Create a test script (example):

```
# Example script name: aer.txt, content:

AER
BUS 1 DEV 0 FN 0        // BUS, DEV, FN correspond to the BDF of the DUT (Device
Under Test)
COR_STATUS BAD_TLP      // Test correctable error: bad TLP
HEADER_LOG 0 1 2 3

AER
BUS 1 DEV 0 FN 0        // BUS, DEV, FN correspond to the BDF of the DUT
UNCOR_STATUS COMP_ABORT // Test uncorrectable error: completion abort
HEADER_LOG 4 5 6 7
```

- For correctable errors, besides `BAD_TLP`, valid types include: `RCVR`, `BAD_DLLP`, `REP_ROLL`, `REP_TIMER`.
- For uncorrectable errors, besides `COMP_ABORT`, valid types include: `TRAIN`, `DLP`, `POISON_TLP`, `FCP`, `COMP_TIME`, `UNX_COM`, `RX_OVE`, `MALF_TLP`, `ECRC`, `UNSUP`.
- A single script can execute various injection configurations. Customize as needed.

5. Execute the test script:

```
root@rk3576-buildroot:/data# ./aer-inject aer.txt
[  459.689869] pcieport 0000:00:00.0: aer_inject: Injecting errors
00000040/00000000 into device 0000:01:00.0
[  459.690302] pcieport 0000:00:00.0: AER: Corrected error message received from
0000:01:00.0
[  459.690376] pci 0000:01:00.0: PCIe Bus Error: severity=Corrected, type=Data
Link Layer, (Receiver ID)
[  459.690404] pci 0000:01:00.0:   device [14e4:449d] error
status/mask=00000040/00002000
[  459.690434] pci 0000:01:00.0:    [ 6] BadTLP
```

10. Kernel Error Injection Test Support

If the PCIe link requires testing the RC endpoint function driver, business model/EP endpoint firmware/duplex hardware IP error tolerance, error injection testing can be enabled to simulate the types of errors that may occur during the interaction between the two parties, and assess the stability of the dual-end software and IP.

Usage method:

```
(1) The following commit needs to be included
commit fe835d5fd3329ba629f8c4290c818ef4b8f9895d
Author: Shawn Lin <shawn.lin@rock-chips.com>
Date:   Wed Sep 4 17:04:37 2024 +0800

    PCI: rockchip: dw: Add fault injection support
    Change-Id: Ib214cc1be565bf16bafb6a847215572f35c43753

(2) The functionality described in the "Kernel Stability Statistical
Information" section of this document needs to be enabled, and the corresponding
controller directory needs to be entered

(3) echo "einj_number enable_or_disable error_type error_number" >
fault_injection
    |------------------|-------------------------------------------------------
---|
    | Value            |         Meaning
     |
    |------------------|-------------------------------------------------------
---|
    |einj_number:      |Error injection group number, only supports 0 to 6,
other values are invalid                        |
    |------------------|-------------------------------------------------------
---|
    |enable_or_disable: |Enable or disable error injection, 0 means disable, 1
means enable, other values are invalid      |
    |------------------|-------------------------------------------------------
---|
    |error_type:       |Error injection type selection, choose the error type
corresponding to the einj_number group number as described in the appendix|
    |------------------|-------------------------------------------------------
---|
    |error_number:     |Error injection quantity, only supports 0 to 255, other
values are invalid                          |
    |------------------|-------------------------------------------------------
---|
    For example, echo "2 1 2 128" > fault_injection, represents enabling einj2
and injecting 128 NAK DLLP packets

(4) Start the PCIe link transmission, for example, NVMe: dd if=/dev/nvme0n1
of=/dev/null bs=1M count=5000

(5) Check if the error occurs: cat err_event
    Rx Recovery Request: 0x1f
    ...
    Tx Nak DLLP: 0x80
```

```
(6) Analyze the dual-end software and hardware to see if there are any
unexpected software or hardware exceptions
```

11. Kernel PMU perf Support

11.1 Software and Configuration

```
(1) The following five commits must be included:
commit 0270f32f207f5682a729c17e977eb87bba83823e
Author: Shuai Xue <xueshuai@linux.alibaba.com>
Date:   Fri Dec 8 10:56:50 2023 +0800

    UPSTREAM: PCI: Move pci_clear_and_set_dword() helper to PCI header
    Change-Id: I35125190a4dd8ba25e6ec14b4712750605c22285

commit 1b627c690ade9a72e3cd488e2e11edffb5d0e879
Author: Shuai Xue <xueshuai@linux.alibaba.com>
Date:   Fri Dec 8 10:56:49 2023 +0800

    UPSTREAM: PCI: Add Alibaba Vendor ID to linux/pci_ids.h
    Change-Id: I86188f119a42548ab777df0449f7d0a933f34d12

commit dcfa6c8947baeac74ab44ea8f03d3831a062c14b
Author: Shuai Xue <xueshuai@linux.alibaba.com>
Date:   Fri Dec 8 10:56:51 2023 +0800

    BACKPORT: drivers/perf: add DesignWare PCIe PMU driver
    Change-Id: I470f4dc2791168760517c77dd31a4dacd7dab591

commit 6cb6a00862fa29f815412634569e2015f86e397a
Author: Shawn Lin <shawn.lin@rock-chips.com>
Date:   Tue Sep 3 16:24:36 2024 +0800

    perf/dwc_pcie: Add support for Rockchip vendor devices
    Change-Id: I6fde80440d2fa058b38a7d927eb846f477812b5f

commit 50cb3fcd18fb9defe23ba95eb3962a287e957166
Author: Shawn Lin <shawn.lin@rock-chips.com>
Date:   Tue Sep 3 16:24:36 2024 +0800

    PCI: rockchip: dw: Add dwc pmu support for rockchip
    Change-Id: Ia27ee055aa3e63deeb7fd646411c3542b7019288

(2) The kernel must enable the CONFIG_PERF_EVENTS configuration.
(3) The system must integrate the perf tool; if not available, it can be
downloaded from the development resource access address mentioned in the
appendix of this document.
```

11.2 Usage Instructions

```
(1) List all configurations supported by DWC PCIe PMU
root@rk3576-buildroot:/# /userdata/perf list | grep dwc_rootport
  dwc_rootport_0/CFG_RCVRY/                          [Kernel PMU event]
#Percentage of time link is in rcvry state
  dwc_rootport_0/L0/                                 [Kernel PMU event]
#Percentage of time link is in L0 state
  dwc_rootport_0/L1/                                 [Kernel PMU event]
#Percentage of time link is in L1 state
  dwc_rootport_0/L1_1/                               [Kernel PMU event]
#Percentage of time link is in L1.1 state
  dwc_rootport_0/L1_2/                               [Kernel PMU event]
#Percentage of time link is in L1.2 state
  dwc_rootport_0/L1_AUX/                             [Kernel PMU event] #Status
not supported by RK
  dwc_rootport_0/RX_L0S/                             [Kernel PMU event]
#Percentage of time RX is in L0s state
  dwc_rootport_0/Rx_CCIX_TLP_Data_Payload/           [Kernel PMU event] #RK does
not support CCIX data statistics
  dwc_rootport_0/Rx_PCIe_TLP_Data_Payload/           [Kernel PMU event] #RX TLP
data volume
  dwc_rootport_0/TX_L0S/                             [Kernel PMU event]
#Percentage of time TX is in L0s state
  dwc_rootport_0/TX_RX_L0S/                          [Kernel PMU event]
#Percentage of time both TX and RX are in L0s state
  dwc_rootport_0/Tx_CCIX_TLP_Data_Payload/           [Kernel PMU event] #RK does
not support CCIX data statistics
  dwc_rootport_0/Tx_PCIe_TLP_Data_Payload/           [Kernel PMU event] #TX TLP
data volume
  dwc_rootport_0/one_cycle/                          [Kernel PMU event] #Not
supported by RK
  dwc_rootport_0/rx_ack_dllp,lane=?/                 [Kernel PMU event] #Number
of DLLPs acknowledged by RX
  dwc_rootport_0/rx_atomic,lane=?/                   [Kernel PMU event] #Not
supported by RK
  dwc_rootport_0/rx_ccix_tlp,lane=?/                 [Kernel PMU event] #Not
supported by RK
  dwc_rootport_0/rx_completion_with_data,lane=?/     [Kernel PMU event] #RX
completion packets with data
  dwc_rootport_0/rx_completion_without_data,lane=?/  [Kernel PMU event] #RX
completion packets without data
  dwc_rootport_0/rx_duplicate_tl,lane=?/             [Kernel PMU event] #Number
of RX/TL dup errors
  dwc_rootport_0/rx_io_read,lane=?/                  [Kernel PMU event] #Number
of ior packets on RX
  dwc_rootport_0/rx_io_write,lane=?/                 [Kernel PMU event] #Number
of iow packets on RX
  dwc_rootport_0/rx_memory_read,lane=?/              [Kernel PMU event] #Number
of memr packets on RX
  dwc_rootport_0/rx_memory_write,lane=?/             [Kernel PMU event] #Number
of memw packets on RX
  dwc_rootport_0/rx_message_tlp,lane=?/              [Kernel PMU event] #Number
of messages received on RX
  dwc_rootport_0/rx_nulified_tlp,lane=?/             [Kernel PMU event] #Number
of TLPs discarded due to errors on RX
  dwc_rootport_0/rx_tlp_with_prefix,lane=?/          [Kernel PMU event] #Number
of TLPs with prefix on RX
  dwc_rootport_0/rx_update_fc_dllp,lane=?/           [Kernel PMU event] #Number
of flow control packets received on RX
```

```
  dwc_rootport_0/tx_ack_dllp,lane=?/                  [Kernel PMU event] #Number
of DLLPs acknowledged by TX
  dwc_rootport_0/tx_atomic,lane=?/                    [Kernel PMU event] #Not
supported by RK
  dwc_rootport_0/tx_ccix_tlp,lane=?/                  [Kernel PMU event] #Not
supported by RK
  dwc_rootport_0/tx_completion_with_data,lane=?/      [Kernel PMU event] #TX
completion packets with data
  dwc_rootport_0/tx_completion_without_data,lane=?/   [Kernel PMU event] #TX
completion packets without data
  dwc_rootport_0/tx_configuration_read,lane=?/        [Kernel PMU event] #Number
of cfg-r packets on TX
  dwc_rootport_0/tx_configuration_write,lane=?/       [Kernel PMU event] #Number
of cfg-w packets on TX
  dwc_rootport_0/tx_io_read,lane=?/                   [Kernel PMU event] #Number
of ior packets on TX
  dwc_rootport_0/tx_io_write,lane=?/                  [Kernel PMU event] #Number
of iow packets on TX
  dwc_rootport_0/tx_memory_read,lane=?/               [Kernel PMU event] #Number
of memr packets on TX
  dwc_rootport_0/tx_memory_write,lane=?/              [Kernel PMU event] #Number
of memw packets on TX
  dwc_rootport_0/tx_message_tlp,lane=?/               [Kernel PMU event] #Number
of messages received on TX
  dwc_rootport_0/tx_nulified_tlp,lane=?/              [Kernel PMU event] #Number
of TLPs discarded due to errors on TX
  dwc_rootport_0/tx_tlp_with_prefix,lane=?/           [Kernel PMU event] #Number
of TLPs with prefix sent by TX
  dwc_rootport_0/tx_update_fc_dllp,lane=?/            [Kernel PMU event] #Number
of flow control packets sent by TX

(2) Start a perf feature, using time-based statistics for RX TLP as an example
 /userdata/perf stat -a -e dwc_rootport_0/Rx_PCIe_TLP_Data_Payload/

(3) Initiate transmission
root@rk3576-buildroot:/# dd if=/dev/nvme0n1 of=/dev/null bs=1M count=5000
dd if=/dev/nvme0n1 of=/dev/null bs=1M count=5000
5000+0 records in
5000+0 records out
5242880000 bytes (5.2 GB, 4.9 GiB) copied, 14.9016 s, 352 MB/s

(4) View statistics
Performance counter stats for 'system wide':
5221423060      dwc_rootport_0/Rx_PCIe_TLP_Data_Payload/     (50.01%)
28.298528222 seconds time elapsed

(5) Similarly, test the data volume of TX TLP, and calculate the average RX/TX
bandwidth
PCIe RX Bandwidth = Rx_PCIe_TLP_Data_Payload / duration
PCIe TX Bandwidth = Tx_PCIe_TLP_Data_Payload / duration

(6) Statistics for Lane events
Since each Lane has the same events, to avoid generating a large amount of
redundant information, it is recommended to specify the Lane ID, for example:
 /userdata/perf stat -a -e dwc_rootport_0/rx_memory_read,lane=1/
```

12. Appendix

12.1 Debugfs Exported Information Analysis Table

Event Symbol	Meaning
EBUF Overflow	-
EBUF Under-run	-
Decode Error	Packet decoding error, signal anomaly
Running Disparity Error	Polarity error, imbalance in the ratio of 0s to 1s in 8bit/10bit encoding, Gen2 signal anomaly
SKP OS Parity Error	SKP sequence parity error, Gen3 signal anomaly
SYNC Header Error	Asynchronous packet header error, signal issue
CTL SKP OS Parity Error	Control SKP sequence parity error, Gen3 signal anomaly
Detect EI Infer	Detection of signal crosstalk
Receiver Error	Receiver end error
Rx Recovery Request	RX signal receives a request from the peripheral, entering recovery state for signal correction
N_FTS Timeout	The peripheral's n_fts does not meet the standard, L0s returns to L0 anomaly and enters recovery state
Framing Error	Frame format decoding error, signal anomaly
Deskew Error	Deskew error, signal anomaly
BAD TLP	Receiving an incorrect TLP packet
LCRC Error	Link CRC, signal anomaly
BAD DLLP	Incorrect DLLP packet, signal anomaly
Replay Number Rollover	Too many accumulated retransmission packets, replay buffer overflow
Replay Timeout	Error packet retransmission timeout
Rx Nak DLLP	Receiving a DLLP from downstream and rejecting its access
Tx Nak DLLP	A DLLP sent downstream is rejected
Retry TLP	Number of retransmitted TLP packets, can indicate error rate
FC Timeout	Flow control update timeout
Poisoned TLP	Reporting an incorrect TLP type, possibly due to link reasons or bit flips in RAM
ECRC Error	End CRC, signal anomaly
Unsupported Request	Receiving a TLP packet of an unsupported request type, possibly access to the peripheral is denied

Event Symbol	Meaning
Completer Abort	The device receives a request from the RC but encounters an internal error, requiring termination of the access request.
Completion Timeout	Reading peripheral-related resources, TLP return timeout
Common event signal status	Reading the current controller's PM mode

12.2 Error Injection Configuration Comparison Table

Group Number einj_number	Group Description	Error Type error_type	Error Meaning
0	CRC Error	0x0	TLP LCRC Error Packet
0	CRC Error	0x1	16b CRC of ACK/NAK DLLP Error Packet
0	CRC Error	0x2	16b CRC of Update-FC DLLP Error Packet
0	CRC Error	0x3	TLP ECRC Error Packet
0	CRC Error	0x4	TLP FCRC Error Packet (128b/130bit Encoding)

Group Number einj_number	Group Description	Error Type error_type	Error Meaning
0	CRC Error	0x5	TX Polarity Error TSOS (128b/130bit Encoding)
0	CRC Error	0x6	TX Polarity Error SKPOS (128b/130bit Encoding)
0	CRC Error	0x8	RX LCRC Error Packet
0	CRC Error	0xb	RX ECRC Error Packet
1	Packet Sequence Error	0x0	TLP SEQ# Error
1	Packet Sequence Error	0x1	DLLP SEQ# Error ACK_NAK_DLLP
2	DLLP Error	0x0	Block Transmission of DLLP ACK/NAK Packet[1]
2	DLLP Error	0x1	Block Transmission of Update FC DLLP Packet[1]
2	DLLP Error	0x2	Always Send NAK DLLP Packet[1]
3	Symbol Error	0x0	Invert sync header (128bit/130bit Encoding)
3	Symbol Error	0x1	TS1 Order Set Error
3	Symbol Error	0x2	TS2 Order Set Error
3	Symbol Error	0x3	FTS Order Set Error
3	Symbol Error	0x4	E-idle Order Set Error
3	Symbol Error	0x5	END/EDB Symbol Error
3	Symbol Error	0x6	STP/SDP Symbol Error
3	Symbol Error	0x7	SKP Order Set Error
4	Flow Control FC Error	0x0	Posted TLP Header Credit
4	Flow Control FC Error	0x1	Non-Posted TLP Header Credit
4	Flow Control FC Error	0x2	Completion TLP Header Credit
4	Flow Control FC Error	0x4	Posted TLP Data Credit
4	Flow Control FC Error	0x5	Non-Posted TLP Data Credit

Group Number einj_number	Group Description	Error Type error_type	Error Meaning
4	Flow Control FC Error	0x6	Completion TLP Data Credit
5	Special TLP Error	0x0	Treat ACK DLLP as NAK DLLP to Generate Duplicate TLP Packets
5	Special TLP Error	0x1	Create Invalid TLP Packets, Original Packets Sent to Retry Process
6	Packet Header Check Error	0x0	Generate TLP Header Error
6	Packet Header Check Error	0x1	Generate Error in the First Set of Four Dwords in TLP Prefix
6	Packet Header Check Error	0x2	Error in the Second Set of Dwords in TLP Prefix

[1] Does not stop based on counting to zero, manual command is required to stop.

12.3 PCIe TX Emphasis Preset Value table

Preset Number	Preshoot(dB)	De-emphasis(dB)
P0	0	-6.0 ± 1.5 dB
P1	0	-3.5 ± 1 dB
P2	0	-4.4 ± 1.5 dB
P3	0	-2.5 ± 1 dB
P4	0	0
P5	1.9 ± 1 dB	0
P6	2.5 ± 1 dB	0
P7	3.5 ± 1 dB	-6.0 ± 1.5 dB
P8	3.5 ± 1 dB	-3.5 ± 1 dB
P9	3.5 ± 1 dB	0
P10	0	x

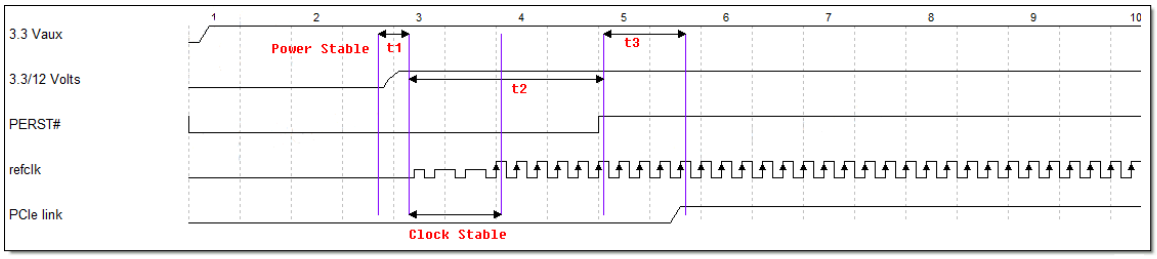
12.4 Development Resource Acquisition Address

After obtaining Redmine permissions, you can directly access https://redmine.rock-chips.com/documents/107 to obtain the various materials mentioned earlier.

12.5 Detailed Explanation of PCIe Address Space Configuration

For detailed configuration of the PCIe address space in DTS, please refer to the following link for interpretation or modification: https://elinux.org/Device_Tree_Usage#PCI_Host_Bridge

12.6 Board-Level Configurable Timing



Adjustable Time	Icon	DTS Property	Description (Linux Kernel Phase)
Power Stable	t1	startup-delay-us	This property is configured in the power node referenced by the vpcie3v3-supply of the PCIe node.
Tpvperl	t2	rockchip,perst-inactive-ms	This property is configured in the PCIe node. If not configured, it defaults to 200ms and takes effect at boot time.
Tpvperl	t2	rockchip,s2r-perst-inactive-ms	This property is configured in the PCIe node. If not configured, it defaults to the same value as rockchip,perst-inactive-ms, and takes effect when resuming from hibernation.
Out of Electrical Idle	t3	rockchip,wait-for-link-ms	This property is configured in the PCIe node. If not configured, it defaults to 1ms and is used to wait for the link to be initiated each time.

12.7 RK3568 PCIe30PHY SRNS Patch

```
diff --git a/drivers/phy/rockchip/phy-rockchip-snps-pcie3.c
b/drivers/phy/rockchip/phy-rockchip-snps-pcie3.c
index 2392bcb..fa3244e 100644
--- a/drivers/phy/rockchip/phy-rockchip-snps-pcie3.c
+++ b/drivers/phy/rockchip/phy-rockchip-snps-pcie3.c
@@ -19,12 +19,15 @@
 #include <dt-bindings/phy/phy.h>

 #define GRF_PCIE30PHY_CON4 0x10
+#define GRF_PCIE30PHY_CON5 0x14
 #define GRF_PCIE30PHY_CON6 0x18

@@ -97,6 +100,17 @@
                 (0x1 << 15) | (0x1 << 31));
```

```
    }

+
+   /* rx0_cmn_refclk_mode disabled */
+   regmap_write(priv->phy_grf, GRF_PCIE30PHY_CON5, (0x0) | (0x1 << 25));
+   /* rx1_cmn_refclk_mode disabled */
+   regmap_write(priv->phy_grf, GRF_PCIE30PHY_CON6, (0x0) | (0x1 << 25));
+
    regmap_write(priv->phy_grf, GRF_PCIE30PHY_CON4,
             (0x0 << 14) | (0x1 << (14 + 16))); //sdram_ld_done
    regmap_write(priv->phy_grf, GRF_PCIE30PHY_CON4,
```